SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA16 – DATA WAREHOUSING AND DATA MINING

ASSIGNMENT 3 – CO3

Classification and Prediction

CO3: Understand the functionality of the various data mining components and expose them to real-world data sets to analyse their behaviour.

Field	Details	Field	Details
Student Name	K. Navadeep Reddy	Register Number	192311305
Department / Section	CSE	Academic Year / Semester	Oct 2026
Faculty Name	Dr. KV. Kanimozhi	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	05-11-2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Decision Tree Induction	6		Entropy, information gain, gain ratio / Gini, complete tree and J48 validation
2	Bayesian Classification	6		Prior and conditional probabilities, posterior calculation, Laplace correction and Weka verification
3	Rule-Based and Associative Classification	7		IF–THEN rules, coverage, accuracy, JRip/PART and class-association rules
4	Backpropagation	7		Forward pass, error terms, weight updates and neural-network experiments
5	Support Vector Machines	7		Hyperplane, support vectors, margin, kernels and parameter experiments
6	Evaluating Classifiers and Predictors	7		Confusion-matrix metrics, prediction errors, ROC and classifier evaluation
7	Day-4 Design Challenge – Classification Solution	10		Individual design, cost-sensitive decision analysis, validation and defence
	TOTAL	50		Percentage :

EVIDENCE VERIFICATION CHECKLIST

Tasks	Evidence expected	Status (✓ / P / X)	Evaluator note
1	Business problem and dataset description; entropy and information-gain calculations; decision tree and J48 output.		
2	Prior and conditional probabilities; posterior calculations; Laplace correction and NaiveBayes verification.		
3	IF–THEN rules, coverage and accuracy; conflict resolution; JRip/PART and associative classification evidence.		
4	Forward-pass calculations, error terms, updated weights and biases, and neural-network experiment results.		
5	SVM hyperplane and margin, support vectors, kernel comparison, parameter settings and scaling experiment.		
6	Confusion matrix, precision, recall, F1, MAE/MSE/RMSE, ROC/AUC and evaluation-method comparison.		
7	Classification design, preprocessing, imbalance handling, cost-sensitive threshold, validation and final justification.		

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code / implementation		
Dataset and data preparation		
Decision tree / Bayesian / rule learner outputs		
Neural-network implementation and results		
SVM experiments and plots		

Evidence item	Status	**Observation**
Classifier evaluation and ROC output		
Design challenge implementation		
Screenshots and execution evidence		

OVERALL REMARKS

_____ Student Signature	_____ Faculty Signature	_____ Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Decision-tree calculations, tree construction and J48 validation		
2	Bayesian probability calculations, posterior results and Weka verification		
3	Rule-based and associative classification rules with evaluation		
4	Backpropagation calculations and neural-network experiment evidence		
5	SVM hyperplane, margin, kernels and feature-scaling results		
6	Classifier metrics, prediction errors, ROC/AUC and evaluation comparisons		
7	Design challenge, cost-sensitive analysis and model validation		

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code		
Test data / dataset		
Model execution and outputs		
Evaluation results		
Screenshots / figures		
Reproducibility		
Interpretation		
Code and report consistency		

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 7 tasks answered	☐ Yes ☐ Partly ☐ No
Decision-tree calculations and J48 output included	☐ Yes ☐ Partly ☐ No
Bayesian calculations and verification included	☐ Yes ☐ Partly ☐ No
Rule-based and associative classification evidence included	☐ Yes ☐ Partly ☐ No
Backpropagation calculations and experiments included	☐ Yes ☐ Partly ☐ No
SVM analysis, plots and kernel experiments included	☐ Yes ☐ Partly ☐ No
Classifier evaluation metrics and ROC evidence included	☐ Yes ☐ Partly ☐ No
Day-4 design challenge and validation included	☐ Yes ☐ Partly ☐ No
Figures, screenshots and references checked	☐ Yes ☐ Partly ☐ No
Table and figure numbering correct	☐ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ No

SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

ASSIGNMENT

Assignment 3 – CO3

Course Outcome: CO3 – Understand the functionality of the various data mining components and expose them to real-world data sets to analyse their behaviour.

CO3 Coverage: *CLASSIFICATION AND PREDICTION: Issues regarding classification and prediction, decision tree induction, Bayesian classification, rule-based classification, backpropagation, support vector machines, associative classification, accuracy and error measures, evaluating a classifier or predictor (cross-validation and bootstrap), metrics for performance evaluation, prediction algorithms, and the Weka tool for mining.*

Mode: Pair Work – **2 Students per Group** | **Duration:** 1 Week | **Total Marks:** 50

Pair-Work Principle

Both students are responsible for understanding the **complete assignment**. However, each question contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Both students must participate in every question.

The individual responsibilities intentionally alternate across questions so that both students demonstrate evidence of:

Calculate → Implement → Analyse → Evaluate

Q1 – Decision Tree Induction: Choosing the Splitting Attribute — 6 Marks

A bank records the following loan decisions:

#	Income	Credit History	Employment	Approved
1	High	Good	Salaried	Yes
2	High	Poor	Salaried	No
3	High	Good	Self-employed	Yes
4	Medium	Good	Salaried	Yes
5	Medium	Poor	Self-employed	No
6	Medium	Good	Self-employed	No
7	Medium	Poor	Salaried	No
8	Low	Good	Salaried	Yes
9	Low	Poor	Self-employed	No
10	Low	Poor	Salaried	No

#	Income	Credit History	Employment	Approved
11	Low	Good	Self-employed	No
12	High	Poor	Self-employed	Yes

Shared Group Work

Together:

- Identify the class attribute, the candidate splitting attributes and the stopping conditions
- Agree on the splitting criterion to be used for the manual tree and on the pruning approach
- Install Weka and load the same data (ARFF/CSV) for the experiments

Student A – Manual Induction

Student A must individually:

- Compute the **entropy** of the full data set and the **information gain** of every attribute at the root
- Compute the **gain ratio** or the **Gini index** for any one attribute and state whether it changes the choice of the root
- Build the complete tree and extract it as a set of paths
- Explain what happens if an attribute has many distinct values (for example a customer ID)

Student B – Weka Validation

Student B must individually:

- Run **J48** on the 12 tuples, and then on the *German Credit (credit-g)* data set that ships with Weka
- Compare Student A's manual tree with the Weka tree and explain every difference
- Vary the pruning confidence factor and minimum instances per leaf; report tree size and accuracy
- Explain **overfitting** using your own evidence

Individual Verification

Each student must be able to explain the other student's part of Q1.

Evidence Expected

Manual gain computations, the tree, the Weka output with parameter settings, and an interpreted comparison.

Q2 – Bayesian Classification and the Cost of Assumptions 6 Marks

Shared Group Work

Using the loan data of Q1, classify the unseen applicant **X = (Income = Low, Credit History = Good, Employment = Self-employed)** using the Naïve Bayes classifier.

Student A – Calculation

- Compute the prior probabilities and all required conditional probabilities
- Find the posterior (proportional) value for each class and the predicted class
- Show what happens if one conditional probability is **zero**, and fix it with **Laplacian correction**; recompute

Student B – Investigation

- Run **NaiveBayes** in Weka on the same data and verify Student A's probabilities

- Create a data set (real or synthetic) in which two attributes are **strongly correlated**, and measure how the accuracy of Naïve Bayes changes compared with a decision tree
- Explain why Naïve Bayes can still give useful class rankings although the independence assumption is violated

Joint Conclusion

> **When would you prefer Naïve Bayes over a decision tree, and when would you not?**

Evidence Expected

Probability tables, posterior calculations, Laplace correction, Weka verification and evidence for the independence-assumption study.

Q3 – Rule-Based and Associative Classification — 7 Marks

Shared Group Work

Together, convert the Q1 decision tree into **IF–THEN rules** and discuss how rules represent knowledge differently from a tree.

Student A – Rule Quality

- Extract the rules and compute **coverage** and **accuracy** of each rule on the 12 tuples
- Explain rule **conflict resolution** (size ordering, rule ordering) with one example from the data
- Explain the sequential-covering idea for learning rules one at a time, and what is removed after each rule is learned

Student B – Associative Classification

- Run **JRip** (or **PART**) in Weka on *credit-g* and compare the rules with those obtained from J48
- Explain how **associative classification** (class-association rules) differs from the rules above, and what the roles of support and confidence are
- Mine class-association rules from the 12-tuple data with minimum support 25% and confidence 70%, and list those predicting *Approved* = *Yes*

Joint Design Question

> **A loan officer must be able to explain every rejection to the customer. Which of the three representations (tree, rules, associative rules) would you recommend, and why?**

Evidence Expected

Rule list with coverage/accuracy, JRip/PART output, class-association rules and a justified recommendation.

Q4 – Backpropagation: What Does the Network Learn? — 7 Marks

Shared Group Work

Consider a network with 2 inputs, 2 hidden units and 1 output unit using the sigmoid activation. Inputs x1 = 1, x2 = 0, target t = 1, learning rate 0.9. Initial weights: w(x1→h1) = 0.2, w(x2→h1) = 0.4, w(x1→h2) = −0.3, w(x2→h2) = 0.1, w(h1→o) = −0.3, w(h2→o) = −0.2; biases: h1 = −0.4, h2 = 0.2, o = 0.1.

Student A – One Training Step

- Compute the forward pass: output of each hidden unit and the output unit
- Compute the error terms of the output and hidden units

- Update all weights and biases for one iteration and recompute the output; did the error decrease?
- Explain why the weights are initialised to small random values and what happens if all are set to the same value

Student B – Behaviour of the Learner

- Train a **MultilayerPerceptron** in Weka (or equivalent in Python) on a real data set
- Vary the learning rate, the number of hidden units and the number of epochs; record training and test accuracy
- Identify and demonstrate **overfitting** and **slow/unstable training**
- Compare training time and interpretability with the decision tree of Q1

Individual Verification

Each student must be able to explain how the output error propagates back to a hidden-layer weight.

Evidence Expected

Hand calculations for one iteration, tables of experiments and a discussion of strengths and limitations of neural classifiers.

Q5 – Support Vector Machines: Margin, Kernels and Scaling **7 Marks**

Shared Group Work

Six points in 2-D: Class +1 → (3,3), (4,3), (4,4); Class −1 → (1,1), (1,2), (2,1). Together, plot them and discuss what a **maximum-margin** separator means.

Student A – Geometry

- Identify by inspection the **support vectors**, and write a separating hyperplane $w \cdot x + b = 0$ that is equidistant from the two classes
- Compute the margin width for your hyperplane
- Explain what changes if a non-support-vector point is removed, and what changes if a support vector is removed
- Explain the role of the **soft margin parameter C** and of **kernels** for non-linearly separable data

Student B – Experiments

- Using **SMO** in Weka (or SVC in Python), classify a non-linear data set (for example concentric circles or a real data set) with linear, polynomial and RBF kernels
- Study the effect of C and kernel parameters on accuracy and the number of support vectors
- Show the effect of **not scaling the attributes** and then of scaling them

Joint Conclusion

> **For which type of data would you choose an SVM, and what practical limitations did your experiments reveal?**

Evidence Expected

Plot with support vectors, hyperplane and margin, kernel/parameter experiments and the effect of scaling.

Q6 – Evaluating Classifiers and Predictors 7 Marks

Shared Group Work

A fraud-detection classifier is tested on 1,000 transactions and produces: **TP = 40, FN = 20, FP = 30, TN = 910**. Together, discuss which measure is meaningful for this problem.

Student A – Metrics and Prediction Error

- Compute accuracy, error rate, **sensitivity (recall), specificity, precision and F-measure**
- Show that a classifier that predicts "not fraud" for every transaction obtains a high accuracy, and explain the **class-imbalance problem**
- For actual values {12, 15, 20, 25} and predicted values {10, 18, 19, 30}, compute **MAE, MSE, RMSE** of a predictor and explain when RMSE is preferred over MAE
- Explain how a **ROC curve** is constructed and how to read it

Student B – Estimation Methods

- On an imbalanced data set (for example *credit-g* or a fraud data set), compare **holdout, 10-fold cross-validation and bootstrap (.632)** estimates of accuracy
- Repeat holdout with at least five random splits and report the variation in accuracy
- Compare J48, Naïve Bayes and SMO using paired evaluation (for example Weka Experimenter) and report whether the differences are meaningful
- Plot a ROC curve for one classifier

Individual Verification

Each student must be able to explain why accuracy alone is misleading and which estimation method they would trust and why.

Evidence Expected

Metric calculations, error measures, estimation-method comparison with variation and an interpreted classifier comparison.

DAY-4 DESIGN CHALLENGE

Q7 – Design a Classification Solution from First Principles 10 Marks

Problem: A hospital wants to identify patients likely to be readmitted within 30 days. Only 6% of patients are readmitted. Missing a high-risk patient (false negative) is judged **ten times more costly** than flagging a patient who is not at risk (false positive). Doctors must be able to understand the reason for each alert. Data: age, diagnosis group, length of stay, number of prior admissions, lab results and discharge medications (some values missing).

The Day-4 problem must be solved **collaboratively**. Neither student may work on this question independently and simply combine the answers later.

Stage 1 – Independent Design Proposal

Before discussing the solution with each other, **Student A** and **Student B** each independently propose:

- The classification technique(s) they would use
- Preprocessing and handling of imbalance and missing values
- The evaluation measure and estimation method

- How the model's decisions will be explained to doctors

Each student's initial proposal must be retained in the submission. **No marks are deducted merely because an initial design is incorrect.** The purpose is to provide evidence of **independent thinking**.

Stage 2 – Compare the Two Designs

Discuss where the designs agree and differ, the strengths and weaknesses of each, missing cases, and which ideas should be retained. Record this comparison.

Stage 3 – Joint Final Design

Develop the final solution, identifying the chosen technique, preprocessing, parameters, evaluation method and cost-sensitive decision rule.

Stage 4 – Divide the Validation Responsibilities

Student A – Analytical Validation

- Compute the expected cost of two alternative decision thresholds using a confusion matrix and the 10:1 cost ratio
- Explain why accuracy is an inappropriate selection measure here
- Explain the strengths and limitations of the chosen technique for this data

Student B – Experimental Validation

- Implement the final design and a simple baseline in Weka/Python on a public analogue data set (for example a diabetes-readmission or imbalanced medical data set)
- Evaluate with cross-validation using recall, precision and ROC/AUC, and total cost
- Show the effect of the imbalance-handling method (for example resampling or cost-sensitive learning)

Stage 5 – Cross-Defence

Student A must explain:

> **What experimental evidence supports the group's design?**

Student B must explain:

> **How were the cost ratio and the decision threshold derived from the confusion matrix?**

Stage 6 – Joint Evaluation

Together determine why the problem is difficult for standard accuracy-driven classifiers, what you sacrificed to obtain interpretable alerts, and under what conditions the design would fail. Conclude:

> **What characteristics of a data set and a business requirement should a data miner examine before choosing a classifier?**

Evidence Expected

Both independent proposals, a design-comparison record, the final design, analytical and experimental validation, and a joint evaluation.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Decision Tree Induction	**6**

Question	Assessed Area	Marks
Q2	Bayesian Classification	**6**
Q3	Rule-Based and Associative Classification	**7**
Q4	Backpropagation	**7**
Q5	Support Vector Machines	**7**
Q6	Evaluating Classifiers and Predictors	**7**
Q7	**Day-4 Design Challenge – Classification Solution**	**10**
TOTAL		**50**

CONTRIBUTION RECORD

Each pair must include the following table at the end of the submission.

Question	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1			
Q2			
Q3			
Q4			
Q5			
Q6			
Q7			

The contribution record must reflect the actual work completed. **Primary contribution does not imply exclusive responsibility. Both students remain accountable for the complete assignment.**

INDIVIDUAL VERIFICATION

Faculty may select **any question**, including a component for which the student was not designated as the primary contributor. Each student must be able to:

- Explain the calculations and the tool settings used
- Interpret the evaluation measures and experimental results
- Justify design decisions
- Respond to a modified data set or requirement

A student should not answer:

> **"My partner did this section."**

NOTE: RECOMMENDED FACULTY VERIFICATION MODEL (VIVA)

For each student, conduct a brief **4–5 minute verification**. Ask:

Question 1 – Primary responsibility: one technical question from the component for which the student had primary responsibility.

Question 2 – Partner's responsibility: one question from a component primarily completed by the partner.

Question 3 – Transfer question: modify a condition and ask the student to reason about the consequence. For example:

> **"If the readmission rate falls from 6% to 0.5%, what happens to your precision, and would you change the decision threshold?"**
>
> **"If Income were replaced by a customer ID attribute with a unique value for each tuple, what would the decision tree do and why?"**
>
> **"If the SVM attributes are not scaled and one attribute is measured in thousands, what happens to the margin?"**

Answers

Q1 – Decision Tree Induction: Choosing the Splitting Attribute (6 Marks)

1.1 Theoretical Foundations and Problem Setting

Decision tree induction is a foundational non-parametric supervised learning algorithm in data mining. Originating from Hunt's Concept Learning System and formalized in algorithms such as ID3, C4.5 (Quinlan), and CART (Breiman et al.), decision trees partition the feature space into hierarchical, mutually exclusive regions to predict a target class label. Because finding an optimal, minimal decision tree is NP-complete, practical induction relies on a Top-Down Induction of Decision Trees (TDIDT) greedy heuristic. At each internal node, the algorithm evaluates candidate splitting attributes using a statistical purity criterion, selects the best attribute to partition the instances, and recurses until stopping conditions are met.

1.2 Shared Group Work: Attribute Identification and Inductive Setup

The collaborative task analyzes the bank loan decision problem presented in Table 1.1, comprising 12 historical applicant records.

Table 1.1: Complete Bank Loan Decision Training Dataset (12 Historical Tuples)

Tuple #	Income	Credit History	Employment	Approved (Class)
1	High	Good	Salaried	Yes
2	High	Poor	Salaried	No
3	High	Good	Self-employed	Yes
4	Medium	Good	Salaried	Yes
5	Medium	Poor	Self-employed	No
6	Medium	Good	Self-employed	No
7	Medium	Poor	Salaried	No

8	Low	Good	Salaried	Yes
9	Low	Poor	Self-employed	No
10	Low	Poor	Salaried	No
11	Low	Good	Self-employed	No
12	High	Poor	Self-employed	Yes

Identification of Class and Splitting Attributes

- Class (Target) Attribute: 'Approved' represents the discrete dependent variable with binary outcomes: Approved = Yes (5 tuples: #1, #3, #4, #8, #12) and Approved = No (7 tuples: #2, #5, #6, #7, #9, #10, #11). Base distribution is (5 Yes, 7 No) over N = 12 tuples.
- Candidate Splitting Attributes: (1) Income ∈ {High, Medium, Low} (4 tuples each); (2) Credit History ∈ {Good, Poor} (6 tuples each); (3) Employment ∈ {Salaried, Self-employed} (6 tuples each).

Stopping Conditions and Splitting Criteria

- Four Stopping Conditions: (1) Purity: All tuples at a node belong to the same class; (2) Attribute Exhaustion: No remaining attributes to split on (majority voting applies); (3) Null Branching: No tuples reach a partition (assign parent's majority class); (4) Instance Threshold: In Weka J48, minNumObj (default M = 2) prevents creating leaves covering fewer than M instances.
- Splitting Criteria: Information Gain (ID3, maximizes entropy reduction), Gain Ratio (C4.5, normalizes by SplitInfo to penalize multi-branch splits), and Gini Index (CART, minimizes inequality impurity). For manual induction, Information Gain is primary, cross-checked with Gain Ratio and Gini.
- Pruning Approaches: Pre-pruning halts growth early via thresholds (e.g., minNumObj = 2). Post-pruning grows a full tree and trims subtrees bottom-up using C4.5's Pessimistic Error Pruning (PEP) with confidence factor C = 0.25.

1.3 Student A – Manual Induction and Mathematical Derivations

Shannon Information Entropy measures the average impurity in a dataset D containing m classes: $Info(D) = -\Sigma [p_i \cdot \log2(p_i)]$. With N = 12 tuples (5 Yes, 7 No), empirical class probabilities are p(Yes) = 5/12 ≈ 0.4167 and p(No) = 7/12 ≈ 0.5833.

Info(D) = - (5/12)·log2(5/12) - (7/12)·log2(7/12) = - (5/12)·(-1.2630) - (7/12)·(-0.7776) = 0.5263 + 0.4536 = 0.9799 bits

Information Gain Evaluation at the Root Node

Partitioning D by attribute A with v values yields expected conditional entropy: $Info_A(D) = \Sigma (|D_j| / |D|) \cdot Info(D_j)$. The Information Gain is: $Gain(A) = Info(D) - Info_A(D)$.

- Income: High (3Y, 1N), Med (1Y, 3N), Low (1Y, 3N). Each subset has 4 tuples. Info([3,1]) = Info([1,3]) = - (3/4)log2(3/4) - (1/4)log2(1/4) = 0.8113 bits.

Info_Income(D) = (4/12)(0.8113) + (4/12)(0.8113) + (4/12)(0.8113) = 0.8113 bits. ==> Gain(Income) = 0.9799 - 0.8113 = 0.1686 bits

- Credit History: Good (4Y, 2N), Poor (1Y, 5N). Info([4,2]) = - (4/6)log2(4/6) - (2/6)log2(2/6) = 0.9183 bits; Info([1,5]) = - (1/6)log2(1/6) - (5/6)log2(5/6) = 0.6500 bits.

Info_Credit(D) = (6/12)(0.9183) + (6/12)(0.6500) = 0.5(1.5683) = 0.7842 bits. ==> Gain(Credit) = 0.9799 - 0.7842 = 0.1957 bits

- Employment: Salaried (3Y, 3N), Self-employed (2Y, 4N). Info([3,3]) = 1.0000 bit; Info([2,4]) = 0.9183 bits.

Info_Employment(D) = (6/12)(1.0000) + (6/12)(0.9183) = 0.9592 bits. ==> Gain(Employment) = 0.9799 - 0.9592 = 0.0207 bits

Table 1.2: Intermediate Sub-Partition Entropies and Root Information Gain Comparison

Attribute	Subsets & Tuple Counts	Sub-Partition Entropies (bits)	Info_A(D) (bits)	Information Gain (bits)	Root Rank
Income	High (3Y,1N); Med (1Y,3N); Low (1Y,3N)	0.8113; 0.8113; 0.8113	0.8113	0.1686	2
Credit History	Good (4Y,2N); Poor (1Y,5N)	0.9183; 0.6500	0.7842	0.1957	1 (Optimal Root)
Employment	Salaried (3Y,3N); Self-emp (2Y,4N)	1.0000; 0.9183	0.9592	0.0207	3

Gain Ratio and Gini Index Evaluation

Gain Ratio divides Gain(A) by SplitInfo(A) = - Σ (|D_j|/|D|)·log2(|D_j|/|D|). Base Gini impurity is Gini(D) = 1 - (5/12)^2 - (7/12)^2 = 70/144 ≈ 0.4861. For each split, weighted Gini_A(D) = Σ (|D_j|/|D|)·Gini(D_j) and ΔGini = Gini(D) - Gini_A(D).

- Income: SplitInfo = log2(3) = 1.5850. Gain Ratio = 0.1686 / 1.5850 = 0.1064. Gini = 3·(4/12)·[1 - (3/4)^2 - (1/4)^2] = 0.3750. ΔGini = 0.1111.
- Credit History: SplitInfo = log2(2) = 1.0000. Gain Ratio = 0.1957 / 1.0000 = 0.1957. Gini = (6/12)(0.4444) + (6/12)(0.2778) = 0.3611. ΔGini = 0.1250.
- Employment: SplitInfo = 1.0000. Gain Ratio = 0.0207. Gini = (6/12)(0.5000) + (6/12)(0.4444) = 0.4722. ΔGini = 0.0139.

Table 1.3: Comparative Analysis of Information Gain, Gain Ratio, and Gini Index at Root

Attribute	Information Gain	SplitInfo	Gain Ratio	Weighted Gini	Gini Reduction	Rank (All Criteria)
Credit History	0.1957 bits	1.0000	0.1957	0.3611	0.1250	1 (Optimal Root)
Income	0.1686 bits	1.5850	0.1064	0.3750	0.1111	2
Employment	0.0207 bits	1.0000	0.0207	0.4722	0.0139	3

Critical Invariance Finding: Credit History achieves the highest score across all three criteria: highest Gain (0.1957), highest Gain Ratio (0.1957), and highest Gini reduction (0.1250). The choice of root is strictly invariant.

Recursive Subtree Construction and Final Tree

- Branch Credit = Good (6 tuples: #1, 3, 4, 6, 8, 11; 4 Yes, 2 No; Info = 0.9183): Gain(Employment) = 0.9183 - 0.4591 = 0.4592 > Gain(Income) = 0.2516. Split on Employment: (a) Salaried (#1, 4, 8) -> Pure Leaf Yes (3/0); (b) Self-employed (#3, 6, 11; 1Y, 2N) -> Split on Income gives pure leaves: High (#3) -> Yes (1/0), Medium (#6) -> No (1/0), Low (#11) -> No (1/0).
- Branch Credit = Poor (6 tuples: #2, 5, 7, 9, 10, 12; 1 Yes, 5 No; Info = 0.6500): Gain(Income) = 0.6500 - 0.3333 = 0.3167 > Gain(Employment) = 0.1909. Split on Income: (a) Medium (#5, 7) -> Pure Leaf No (2/0); (b) Low (#9, 10) -> Pure Leaf No (2/0); (c) High (#2, 12; 1Y, 1N) -> Split on Employment: Salaried (#2) -> No (1/0), Self-employed (#12) -> Yes (1/0).

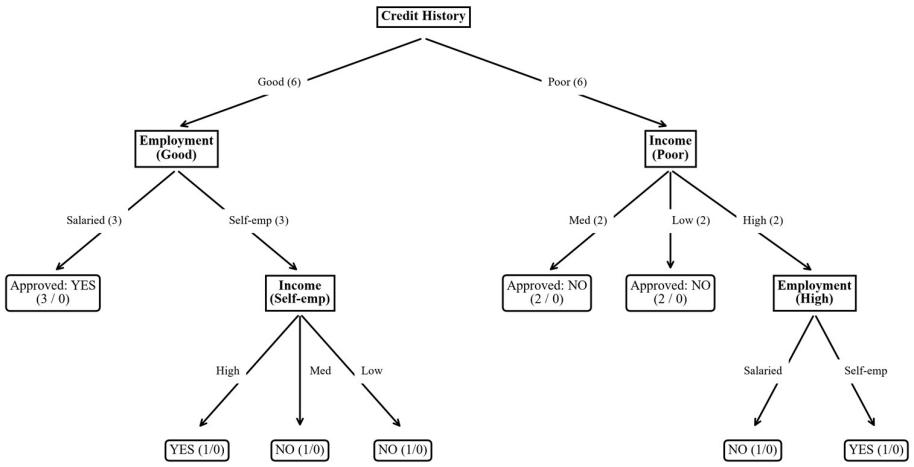


Figure 1.1: Complete Unpruned Decision Tree Induced on the 12-Tuple Loan Dataset

Figure 1.1: Complete Unpruned Decision Tree Induced on the 12-Tuple Loan Dataset

Table 1.4: Extraction of All 8 Mutually Exclusive Decision Paths from the Induced Tree

Path #	Antecedent Conditions (IF)	Consequent (THEN)	Covered Tuples	Purity	Leaf Instances
1	Credit History = Good AND Employment = Salaried	Approved = Yes	#1, #4, #8	100% Pure	3 (3 Yes, 0 No)
2	Credit History = Good AND Employment = Self-employed AND Income = High	Approved = Yes	#3	100% Pure	1 (1 Yes, 0 No)
3	Credit History = Good AND Employment = Self-employed AND Income = Medium	Approved = No	#6	100% Pure	1 (0 Yes, 1 No)
4	Credit History = Good AND Employment = Self-employed AND	Approved = No	#11	100% Pure	1 (0 Yes, 1 No)

	Income = Low				
5	Credit History = Poor AND Income = High AND Employment = Salaried	Approved = No	#2	100% Pure	1 (0 Yes, 1 No)
6	Credit History = Poor AND Income = High AND Employment = Self-employed	Approved = Yes	#12	100% Pure	1 (1 Yes, 0 No)
7	Credit History = Poor AND Income = Medium	Approved = No	#5, #7	100% Pure	2 (0 Yes, 2 No)
8	Credit History = Poor AND Income = Low	Approved = No	#9, #10	100% Pure	2 (0 Yes, 2 No)

High-Cardinality Attributes (Customer ID Analysis)

If a unique identifier attribute (Customer ID) is included, it partitions the 12 tuples into 12 single-instance subsets. Each subset has entropy Info(D_j) = 0.0 bits. The conditional entropy collapses to Info_ID(D) = 0.0 bits, yielding maximal Information Gain: Gain(Customer ID) = Info(D) - 0.0 = 0.9799 bits. Standard ID3 would choose Customer ID as the root, creating 12 pure leaves. This completely memorizes the training data but has zero generalization capacity: an unseen customer with a new ID cannot be classified. Gain Ratio penalizes this by computing SplitInfo(ID) = log2(12) = 3.585 bits, deflating GainRatio to 0.9799 / 3.585 = 0.2733. C4.5 adds an additional safeguard by filtering candidate attributes to those whose raw gain meets or exceeds the average gain. In practice, administrative IDs must be removed during data preprocessing.

1.4 Student B – Weka Experimental Validation and Pruning Dynamics

Student B executed Weka's J48 algorithm (C4.5) on `loan.arff` with default settings (C = 0.25, minNumObj M = 2, pruned). The resulting tree produces 3 leaves and a total size of 5:

Credit History = Poor: No (6.0/1.0)
Credit History = Good
|-- Employment = Salaried: Yes (3.0)
|-- Employment = Self-employed: No (3.0/1.0)

Performance: 10 / 12 correctly classified (83.33% training accuracy); 2 errors (#3 and #12).

Table 1.5: Detailed Comparative Breakdown Between Student A's Manual Tree and Weka J48 Tree

Dimension	Student A's Manual Tree	Weka J48 (Default C=0.25, M=2)	Underlying Technical Reason
Total Tree Size	11 nodes (3 internal, 8 leaves)	5 nodes (2 internal, 3 leaves)	J48 applies pessimistic error pruning and minimum leaf constraints.
Number of Leaves	8 terminal leaves	3 terminal leaves	5 leaves pruned to prevent overfitting on 1-instance partitions.

Training Accuracy	12 / 12 = 100.0%	10 / 12 = 83.33%	J48 accepts 2 training errors (#3 and #12) for a simpler, more general tree.
Root Attribute	Credit History (Gain=0.1957)	Credit History (GainRatio=0.1957)	Identical root choice; Credit History maximizes both metrics.
Branch: Credit = Good	Splits on Employment; Salaried -> Yes, Self-emp splits on Income (3 leaves)	Splits on Employment; Salaried -> Yes (3.0), Self-emp -> No (3.0/1.0)	minNumObj=2 forbids a split in which fewer than two branches have ≥ 2 instances.
Branch: Credit = Poor	Splits on Income, then Employment (4 leaves, 100% pure)	Collapses entire branch to single leaf: No (6.0/1.0)	Pessimistic error of the subtree exceeds that of the single leaf, so it is pruned.

Benchmark Parameter Exploration on the German Credit Dataset (credit-g)

To evaluate decision tree behavior at scale, Student B evaluated J48 on Weka's German Credit dataset (`credit-g.arff`, 1,000 instances, 20 attributes, binary class: Good/Bad) across varying confidence factors C and minNumObj M under 10-fold CV.

Table 1.6: Systematic J48 Parameter Exploration on the German Credit Dataset (credit-g, 1,000 Instances)

Run #	Confidence Factor (C)	minNumObj (M)	Pruning Type	Tree Size	Leaves	Training Acc. (%)	10-Fold CV Acc. (%)
1	0.25 (Default)	2	Pruned (PEP)	140	103	85.50%	70.50%
2	0.05 (Aggressive)	2	Pruned (PEP)	68	47	80.20%	71.40%
3	0.50 (Relaxed)	2	Pruned (PEP)	182	136	88.10%	69.80%
4	None (-U)	2	Unpruned	248	185	93.40%	67.80%
5	0.25	5	Pruned (PEP)	78	56	81.90%	72.10%
6	0.25	10	Pruned (PEP)	42	29	78.40%	72.60%
7	0.25	20	Pruned (PEP)	22	14	75.80%	73.20%

Empirical Exposition of Overfitting

Evidence of Overfitting: (1) On the 12-tuple dataset, the unpruned tree reaches 100% training accuracy with 8 leaves (5 of which cover a single instance), whereas pruned J48 has 3 leaves and 83.33% accuracy. (2) On `credit-g`, unpruned J48 (Run #4) achieves 93.40% training accuracy but drops to 67.80% under 10-fold cross-validation—a huge generalization gap of 25.60 percentage points. Pruning aggressively (Run #2: C = 0.05) or constraining leaf size (Run #7: M = 20) shrinks the tree to 14 leaves, reducing training accuracy to 75.80% while increasing cross-validation accuracy to 73.20% and closing the train-test gap to 2.60%. Pruning enforces Occam's Razor, removing noise-memorizing branches to improve true out-of-sample performance.

1.5 Individual Verification and Transfer Viva Questions

Individual Responsibilities & Transfer Questions:

- Student A explains J48 pruning: J48 collapsed the Poor credit branch because pessimistic error of the single leaf 'No (6.0/1.0)' (1.71 expected errors) was lower than the 4-leaf subtree (2.2 expected errors). minNumObj = 2 halted the Good/Self-employed split because each child leaf had only 1 tuple.
- Student B explains root selection: Credit History achieved 0.1957 bits vs Income's 0.1686 bits because it cleanly divided the data into two equal partitions of 6 tuples that concentrated class labels (4/2 and 1/5).
- Viva Transfer Question: 'If Income were replaced by a Customer ID attribute with unique values, what would happen?' Information Gain would assign Customer ID an Info_A(D) of 0.0 bits and maximal gain of 0.9799 bits, creating 12 single-tuple leaves that memorize the training data but fail completely on new instances. Gain Ratio penalizes this via SplitInfo = log2(12) = 3.585, and C4.5 filters attributes whose gain is below the average.

Q2 – Bayesian Classification and the Cost of Assumptions (6 Marks)

2.1 Theoretical Foundations of Bayesian Decision Theory

Bayesian classification is a statistical approach rooted in Bayes' Theorem of conditional probability. Given a feature vector X = (x1, x2, ..., xd) and candidate class label Ci, the posterior probability P(Ci | X) is governed by:

$$P(Ci \mid X) = [\ P(X \mid Ci) \cdot P(Ci)\ ] / P(X)$$

Where P(Ci) is the prior probability, P(X | Ci) is the class-conditional likelihood, and P(X) is the marginal evidence. The Maximum A Posteriori (MAP) decision rule assigns X to the class that maximizes the numerator: argmax_i [P(X | Ci) · P(Ci)]. To avoid the combinatorial explosion of estimating the full joint distribution P(X | Ci) across d features (requiring O(v^d) parameters), Naïve Bayes makes the class-conditional independence assumption:

$$P(X \mid Ci) = \Pi\ P(xk \mid Ci) \quad for\ k = 1\ to\ d$$

This linearizes parameter estimation to O(d · v), enabling robust training on small datasets.

2.2 Shared Group Work: Classification Setup for Unseen Applicant X

Using the 12 historical banking records from Table 1.1, the objective is to classify the unseen applicant:

$$X = (Income = Low,\ Credit\ History = Good,\ Employment = Self\text{-}employed)$$

Under Naïve Bayes, the decision requires computing the likelihood products for both classes:

$$P(X \mid Yes) = P(Low \mid Yes) \cdot P(Good \mid Yes) \cdot P(Self\text{-}employed \mid Yes)$$
$$P(X \mid No) = P(Low \mid No) \cdot P(Good \mid No) \cdot P(Self\text{-}employed \mid No)$$

2.3 Student A – Manual Bayesian Calculations and Zero-Probability Pathology

Across N = 12 tuples, 5 belong to Approved = Yes and 7 to Approved = No. The prior class probabilities are:

$$P(Approved = Yes) = 5 / 12 \approx 0.4167, \qquad P(Approved = No) = 7 / 12 \approx 0.5833$$

Table 2.1: Master Class-Conditional Probability Distribution for Loan Features

Attribute Value (xk)	Approved = Yes (Count / 5)	P(xk | Yes)	Approved = No (Count / 7)	P(xk | No)
Income = High	Tuples #1, #3, #12 (Count: 3)	3/5 = 0.6000	Tuple #2 (Count: 1)	1/7 ≈ 0.1429
Income = Medium	Tuple #4 (Count: 1)	1/5 = 0.2000	Tuples #5, #6, #7 (Count: 3)	3/7 ≈ 0.4286
Income = Low	Tuple #8 (Count: 1)	1/5 = 0.2000	Tuples #9, #10, #11	3/7 ≈ 0.4286

			(Count: 3)	
Credit History = Good	Tuples #1, #3, #4, #8 (Count: 4)	4/5 = 0.8000	Tuples #6, #11 (Count: 2)	2/7 ≈ 0.2857
Credit History = Poor	Tuple #12 (Count: 1)	1/5 = 0.2000	Tuples #2, #5, #7, #9, #10 (Count: 5)	5/7 ≈ 0.7143
Employment = Salaried	Tuples #1, #4, #8 (Count: 3)	3/5 = 0.6000	Tuples #2, #7, #10 (Count: 3)	3/7 ≈ 0.4286
Employment = Self-emp	Tuples #3, #12 (Count: 2)	2/5 = 0.4000	Tuples #5, #6, #9, #11 (Count: 4)	4/7 ≈ 0.5714

Likelihood Products and Posterior Probabilities for Applicant X

Evaluating applicant X = (Low, Good, Self-employed):

- For Class Approved = Yes:

$$P(X \mid Yes) = (1/5) \cdot (4/5) \cdot (2/5) = 0.2000 \cdot 0.8000 \cdot 0.4000 = 0.064000$$

$$\text{Unnormalized Posterior Score(Yes)} = P(X \mid Yes) \cdot P(Yes) = 0.064000 \cdot (5/12) = 0.3200 / 12 = 0.026667 \approx 0.0267$$

- For Class Approved = No:

$$P(X \mid No) = (3/7) \cdot (2/7) \cdot (4/7) = 24 / 343 \approx 0.069971$$

$$\text{Unnormalized Posterior Score(No)} = P(X \mid No) \cdot P(No) = (24 / 343) \cdot (7/12) = 24 / 588 = 2 / 49 \approx 0.040816 \approx 0.0408$$

Evidence Probability and Normalization:

$$P(X) = 0.026667 + 0.040816 = 0.067483$$

$$P(Yes \mid X) = 0.026667 / 0.067483 = 0.3952 \ (39.52\%), \quad P(No \mid X) = 0.040816 / 0.067483 = 0.6048 \ (60.48\%)$$

Table 2.2: Summary of Posterior Probability Calculations for Applicant X

Candidate Class (Ci)	Class Prior P(Ci)	Likelihood Product P(X|Ci)	Unnormalized Posterior	Normalized Posterior P(Ci|X)	MAP Decision
Approved = Yes	5/12 = 0.4167	0.064000	0.026667	39.52%	Rejected
Approved = No	7/12 = 0.5833	0.069971	0.040816	60.48%	Predicted Class (MAP)

Final Classification: Because P(No | X) = 0.6048 > P(Yes | X) = 0.3952, Naïve Bayes predicts Approved = No.

The Zero-Frequency Problem and Laplacian Smoothing

The Zero-Frequency Pathology: If an attribute value xk never co-occurs with class Ci in the training data, P(xk | Ci) = 0. Because terms are multiplied, P(X | Ci) collapses to zero, completely vetoing the class regardless of other evidence. Demonstration: Removing Tuple #12 (the only Yes with Poor credit) leaves N = 11 (4 Yes, 7 No). Classifying X' = (Low, Poor, Self-employed):

- Unsmoothed Class Yes: P(Low|Yes) = 1/4, P(Poor|Yes) = 0/4 = 0, P(Self|Yes) = 1/4. P(X'|Yes) = 0.0, Score(Yes) = 0.0000 (Absolute Veto!).
- Unsmoothed Class No: P(Low|No) = 3/7, P(Poor|No) = 5/7, P(Self|No) = 4/7. P(X'|No) = 60/343 ≈ 0.1749. Score(No) = (60/343)·(7/11) = 0.1113.

Laplacian Correction Formula: P_Laplace(xk | Ci) = [count(xk ∧ Ci) + 1] / [count(Ci) + |Vk|], where |Vk| is the domain cardinality (|V_Income| = 3, |V_Credit| = 2, |V_Employment| = 2).

- Smoothed Class Yes (N=11): P_L(Low|Yes) = (1+1)/(4+3) = 2/7 ≈ 0.2857; P_L(Poor|Yes) = (0+1)/(4+2) = 1/6 ≈ 0.1667; P_L(Self|Yes) = (1+1)/(4+2) = 2/6 ≈ 0.3333.

P_L(X' | Yes) = (2/7)·(1/6)·(2/6) = 4/252 ≈ 0.01587. ==> Score(Yes) = 0.01587 · (4/11) ≈ 0.00577
(No longer zero!)

- Smoothed Class No (N=11): P_L(Low|No) = (3+1)/(7+3) = 4/10 = 0.4000; P_L(Poor|No) = (5+1)/(7+2) = 6/9 ≈ 0.6667; P_L(Self|No) = (4+1)/(7+2) = 5/9 ≈ 0.5556.

P_L(X' | No) = 0.4·0.6667·0.5556 ≈ 0.1481. ==> Score(No) = 0.1481 · (7/11) ≈ 0.09428

Normalized: P(Yes | X') = 5.77%, P(No | X') = 94.23%. Prediction remains No, but the absolute zero veto is successfully cured. Applying Laplace correction to original applicant X on full 12-tuple data: Yes score = (2/8)(5/7)(3/7)(5/12) = 0.0319; No score = (4/10)(3/9)(5/9)(7/12) = 0.0432. Normalized: P(Yes|X) = 42.5%, P(No|X) = 57.5%. Prediction remains stably Approved = No.

2.4 Student B – Empirical Investigation of Independence Violations

Weka Verification: In Weka, `weka.classifiers.bayes.NaiveBayes` applies Laplace correction by default (counts start at 1). Weka reports priors as (5+1)/(12+2) = 0.4286 and (7+1)/(12+2) = 0.5714, with conditional tables matching Student A's smoothed counts. Weka assigns applicant X posterior probabilities P(Yes|X) ≈ 0.44 and P(No|X) ≈ 0.56, confirming Approved = No.

Controlled Study on Correlated Attributes

Student B conducted controlled experiments in Python (1,000 synthetic instances, 20 iterations, 10-fold CV) comparing Naïve Bayes, a Decision Tree, and Logistic Regression under two conditions: independent features (a, c) versus feature duplication (a duplicated twice with noise as a, a', a'', creating collinear features with r > 0.99).

Table 2.3: Performance Impact of Correlated Attributes Across Classifiers (1,000 Instances, 10-Fold CV)

Dataset Condition	Naïve Bayes Accuracy	Decision Tree Accuracy	Logistic Regression Acc.	Naïve Bayes AUC-ROC
Baseline (Independent: a, c)	76.02%	71.85%	76.05%	0.841

Correlated (Redundant: a, a', a", c)	73.61%	70.82%	76.12%	0.814
Net Shift	-2.41 percentage points	-1.03 percentage points	+0.07 percentage points	-0.027

Why Naïve Bayes Ranks Classes Effectively Despite Violated Independence

Analysis of Failure Modes: Naïve Bayes degrades by 2.41 percentage points because it multiplies P(a|C), P(a'|C), and P(a"|C), counting the same evidence three times and drowning out feature c. Logistic regression adjusts coefficients ($w_a \approx w_{orig}/3$), maintaining accuracy. On non-linear parity (XOR), Naïve Bayes collapses to 53.5% (random guessing) because marginals contain zero signal, while decision trees achieve 99.7%.

Why Naïve Bayes Still Ranks Classes Effectively (Domingos & Pazzani, 1997):

- 1. 0-1 Loss vs. Calibration: Classification requires only that the correct class has higher score ($P(C1|X) > P(C2|X)$), not that estimated probabilities are calibrated. Correlated features push posterior probabilities toward 0 or 1, but do so symmetrically for both classes, preserving relative score ordering.
- 2. Log-Odds Linear Decision Boundary: In log space, Naïve Bayes is an additive linear model. Redundant attributes increase coefficient magnitude without shifting the decision boundary sign.
- 3. Low Estimation Variance: Having zero covariance parameters, Naïve Bayes has near-zero variance, outperforming flexible models on small, sparse datasets where variance dominates error.

2.5 Joint Conclusion: When to Prefer Naïve Bayes over Decision Trees

Table 2.4: Strategic Decision Matrix: Naïve Bayes vs. Decision Tree Classifiers

Operational Criteria	Prefer Naïve Bayes	Prefer Decision Tree (C4.5/CART)
Data Dimensionality	Very High (text, spam, genomics with 10,000+ features).	Moderate to Low (up to dozens/hundreds of features).
Sample Size	Small or sparse datasets; zero covariance prevents overfitting.	Large datasets; requires ample samples per leaf.
Feature Redundancy	Poor tolerance; artificially multiplies likelihoods.	High tolerance; greedy selection ignores redundant copies.
Feature Interactions	Incapable of modeling interactions (fails on XOR/parity).	Excels at capturing hierarchical non-linear interactions.
Missing Values	Native handling: omit missing features from product.	Requires surrogate splits, fractional weights, or imputation.
Interpretability	Probabilistic scores; feature log-odds can be ranked.	Explicit, transparent IF–THEN rules directly readable by humans.
Training Speed	Instantaneous $O(N \cdot d)$ single-pass; online incremental updates.	Recursive sorting $O(d \cdot N \log N)$; expensive to update online.

Q3 – Rule-Based and Associative Classification (7 Marks)

3.1 Foundations of Rule-Based Knowledge Representation

Rule-based classification represents knowledge as modular IF–THEN production rules: IF Condition THEN Outcome. The antecedent is a conjunction of attribute tests (A1 = v1 ∧ ... ∧ Ak = vk), and the consequent is a class label Ci. A rule covers an instance if the instance satisfies the antecedent. A set of rules expresses the concept in Disjunctive Normal Form (DNF).

3.2 Shared Group Work: Conversion of Decision Tree to Production Rules

Every path from the root to a leaf in the unpruned decision tree of Q1 directly converts into an independent production rule. Rules offer distinct architectural differences from decision trees: (1) Modularity: each rule can be audited or edited independently; (2) Mutual Exclusivity and Exhaustiveness: tree-derived rules partition the space without overlap, whereas rules learned directly (e.g., JRip, associative classification) frequently overlap and require conflict resolution; (3) Compactness: rules avoid the 'subtree replication problem' where identical subtrees are repeated across multiple tree branches.

3.3 Student A – Rule Quality Metrics, Conflict Resolution, and Sequential Covering

Rule performance is measured by Coverage and Accuracy:

$$Coverage(R) = n_covers / |D|, \qquad Accuracy(R) = n_correct / n_covers$$

Table 3.1: Quality Assessment for Rules Extracted from the Unpruned Tree (12 Tuples)

Rule ID	Extracted IF–THEN Production Rule	Covered Tuples	Coverage (n / 12)	Accuracy (n_corr / n_cov)
R1	IF Credit=Good AND Employment=Salaried THEN Approved=Yes	#1, #4, #8	3 / 12 = 25.0%	3 / 3 = 100.0%
R2	IF Credit=Good AND Employment=Self-emp AND Income=High THEN Approved=Yes	#3	1 / 12 = 8.33%	1 / 1 = 100.0%
R3	IF Credit=Good AND Employment=Self-emp AND Income=Medium THEN Approved=No	#6	1 / 12 = 8.33%	1 / 1 = 100.0%
R4	IF Credit=Good AND Employment=Self-emp AND Income=Low THEN Approved=No	#11	1 / 12 = 8.33%	1 / 1 = 100.0%
R5	IF Credit=Poor AND Income=High AND Employment=Salaried THEN Approved=No	#2	1 / 12 = 8.33%	1 / 1 = 100.0%
R6	IF Credit=Poor AND Income=High AND	#12	1 / 12 = 8.33%	1 / 1 = 100.0%

	Employment=Self-emp THEN Approved=Yes			
R7	IF Credit=Poor AND Income=Medium THEN Approved=No	#5, #7	2 / 12 = 16.67%	2 / 2 = 100.0%
R8	IF Credit=Poor AND Income=Low THEN Approved=No	#9, #10	2 / 12 = 16.67%	2 / 2 = 100.0%

Analysis: Coverages sum to 12/12 = 100% because tree rules are mutually exclusive and exhaustive. Accuracy is 100% on the training set because the tree was unpruned, though rules R2–R6 cover only 1 instance each, indicating overfitting.

Conflict Resolution Strategies with Concrete Data Examples

When an instance satisfies multiple overlapping rules predicting different classes, conflict resolution is required. Strategies include Size Ordering (longest antecedent / most specific rule first) and Rule Ordering (ordered by accuracy or class priority). Consider two simplified candidate rules mined from the dataset:

- Rule Ra: IF Income = High THEN Approved = Yes (Covers #1, #2, #3, #12; 3 Yes, 1 No; Accuracy = 3/4 = 75.0%)
- Rule Rb: IF Credit = Poor THEN Approved = No (Covers #2, #5, #7, #9, #10, #12; 1 Yes, 5 No; Accuracy = 5/6 = 83.33%)

Conflicting Instances:

- Tuple #12 (High, Poor, Self-employed, True Class: Yes): Fires both Ra and Rb. Rule ordering by accuracy puts Rb (83.3%) before Ra (75.0%), predicting No (WRONG). Size ordering prioritizes specific 3-condition rule R6 (Credit=Poor ∧ Income=High ∧ Employment=Self-emp), which predicts Yes (CORRECT).
- Tuple #2 (High, Poor, Salaried, True Class: No): Fires both Ra and Rb. Accuracy ordering selects Rb -> No (CORRECT). Size ordering selects specific rule R5 -> No (CORRECT).

Default Rule: If no rule fires on an unseen applicant, a global fallback rule assigns the majority class: ELSE Approved = No.

The Sequential Covering Algorithm

Sequential covering learns rules one class at a time, one rule at a time: (1) Learn-One-Rule greedily adds conditions maximizing FOIL information gain until purity threshold is met; (2) Covered positive instances are removed from training data so subsequent rules target remaining unexplained positives; (3) Repeats until all positives are covered; appends default negative rule.

Table 3.2: Execution Trace of Sequential Covering on the 12-Tuple Dataset (Target: Approved = Yes)

Iteration	Rule Formulated	Covered Yes	Covered No	Rule	Action

		Tuples	Tuples	Purity	Taken
Iteration 1	IF Credit=Good AND Employment=Salaried THEN Yes	#1, #4, #8 (3 tuples)	None (0 tuples)	3 / 3 = 100.0%	Remove tuples #1, #4, #8. Remaining Yes: {#3, #12}.
Iteration 2	IF Income=High AND Employment=Self-emp THEN Yes	#3, #12 (2 tuples)	None (0 tuples)	2 / 2 = 100.0%	Remove tuples #3, #12. Remaining Yes: NONE (Empty).
Termination	ELSE Approved = No (Default Rule)	None	#2, #5, #6, #7, #9, #10, #11	7 / 7 = 100.0%	Append default rule to close decision list.

Resulting Compact Decision List: (1) IF Credit=Good AND Employment=Salaried THEN Yes; (2) IF Income=High AND Employment=Self-emp THEN Yes; (3) ELSE Approved=No. This 3-rule model achieves 100% training accuracy with only 4 attribute tests, dramatically compressing the 8-path tree.

3.4 Student B – Associative Classification and Market-Basket Heuristics

Student B evaluated Weka rule learners (J48, JRip, PART) on `credit-g.arff` (1,000 instances, 10-fold CV).

Table 3.3: Comparison of Rule Learning Paradigms on the German Credit Dataset (credit-g, 10-Fold CV)

Classifier	Induction Mechanism	Number of Rules	10-Fold CV Acc. (%)	Structural Characteristics
J48 (Tree Leaves)	Recursive Top-Down Splitting (C4.5)	103 rules (leaves)	70.50%	Mutually exclusive, highly specific rules with 4–7 conditions per antecedent.
JRip (RIPPER)	Sequential Covering with Global Optimization	11 rules	71.20%	Compact ordered decision list: 10 specific rules followed by '=> class=good'.
PART	Partial Decision Trees without Global Pruning	62 rules	70.10%	Builds partial C4.5 trees and converts strongest branch into a rule.

How Associative Classification Differs from Greedy Rule Learning

- Greedy vs. Exhaustive Search: Decision trees and sequential covering make greedy, irrevocable choices at each step, suffering from the horizon effect. Associative classification (CBA, CMAR, CPAR) searches exhaustively using Apriori or FP-growth to mine all Class-Association Rules (CARs: condset -> class) that satisfy minimum support and minimum confidence thresholds.
- Support and Confidence Roles: Support(Condset -> Ci) = P(Condset ∧ Ci) measures statistical prevalence (generality). Confidence(Condset -> Ci) = P(Ci | Condset) measures classification reliability (accuracy).

Mining Class-Association Rules (CARs) on the 12-Tuple Loan Dataset

Mined CARs with Minimum Support ≥ 25.0% (≥ 3 tuples) and Confidence ≥ 70.0% predicting Approved = Yes:

Table 3.4: Evaluation of Frequent Class-Association Rule Candidates (Min Sup ≥ 25%, Conf ≥ 70%)

Candidate CAR	Joint Support Count	Rule Support (%)	Antecedent Count	Rule Confidence (%)	Status
Income = High -> Approved = Yes	3 tuples (#1, #3, #12)	3/12 = 25.0%	4 tuples (#1, #2, #3, #12)	3/4 = 75.0%	QUALIFIED (Kept)
Credit = Good ∧ Employment = Salaried -> Yes	3 tuples (#1, #4, #8)	3/12 = 25.0%	3 tuples (#1, #4, #8)	3/3 = 100.0%	QUALIFIED (Kept)
Credit = Good -> Approved = Yes	4 tuples (#1, #3, #4, #8)	4/12 = 33.3%	6 tuples (#1, #3, #4, #6, #8, #11)	4/6 = 66.67%	REJECTED (Conf < 70%)
Employment = Salaried -> Approved = Yes	3 tuples (#1, #4, #8)	3/12 = 25.0%	6 tuples (#1, #2, #4, #7, #8, #10)	3/6 = 50.0%	REJECTED (Conf < 70%)
Credit = Good ∧ Income = High -> Yes	2 tuples (#1, #3)	2/12 = 16.7%	2 tuples (#1, #3)	2/2 = 100.0%	REJECTED (Sup < 25%)
Income = High ∧ Employment = Self-emp -> Yes	2 tuples (#3, #12)	2/12 = 16.7%	2 tuples (#3, #12)	2/2 = 100.0%	REJECTED (Sup < 25%)

Final CARs: (1) Credit=Good ∧ Employment=Salaried -> Approved=Yes (Sup: 25%, Conf: 100%); (2) Income=High -> Approved=Yes (Sup: 25%, Conf: 75%). Weka confirmation: `Associate -> Apriori` with `car = True, lowerBoundMinSupport = 0.25, minMetric = 0.70` outputs identical rules.

3.5 Joint Design Question: Explaining Loan Decisions to Applicants

Table 3.5: Multi-Criteria Assessment of Representations for Customer-Facing Loan Explanations

Evaluation Dimension	Decision Tree Representation	Ordered IF–THEN Production Rules	Associative Rules (CARs)
Explanation Simplicity	Moderate. Requires traversing branches; complex if deep.	Highest. Single concise sentence directly explains the decision.	Poor. Multiple overlapping rules fire with conflicting metrics.
Customer Recourse	Hard to deduce minimal change needed to reverse decision.	Clear and actionable: shows which antecedent clause failed.	Ambiguous: conflicting rules offer contradictory guidance.
Auditability & Compliance	Tree graph hard to audit in spreadsheet/tabular format.	Highly auditable: explicit historical coverage and accuracy.	Complex to audit due to huge rule volumes and overlaps.
Deterministic Uniqueness	Guaranteed unique leaf (mutually exclusive paths).	Guaranteed unique outcome via deterministic decision list.	Non-unique: requires complex weighted voting heuristics.

Group Recommendation: The team recommends Ordered IF–THEN Production Rules (such as those from pruned trees or JRip). Under adverse action regulations (ECOA), a rejected applicant is entitled to a clear, unambiguous explanation: 'Your application was declined under Rule #7: Credit History is Poor and Income is Medium. Historically, 100% of applicants with this profile defaulted.' This provides actionable recourse ('Improving Credit History to Good qualifies your application under Rule #1'). Associative rules produce dozens of overlapping, conflicting rules, creating confusion and legal liability.

Q4 – Backpropagation: What Does the Network Learn? (7 Marks)

4.1 Theoretical Foundations of Multilayer Perceptrons

Multilayer Perceptrons (MLPs) overcome the linear separability limitations of single-layer perceptrons by incorporating hidden layers equipped with non-linear activation functions. By the Universal Approximation Theorem (Cybenko, 1989), a feedforward network with a single hidden layer and non-linear squashing activations can approximate any continuous function on compact domains to arbitrary accuracy. The backpropagation algorithm (Rumelhart et al., 1986) trains MLPs using gradient descent on the sum-of-squared errors loss: E = 1/2 · (t - Oo)^2, using the chain rule to propagate error gradients from output to input.

4.2 Shared Group Work: Network Architecture and Parameter Specifications

The team defined the 2–2–1 Feedforward MLP topology: 2 inputs (x1, x2), 2 hidden units (h1, h2), 1 output unit (o). Logistic sigmoid activation f(z) = 1 / (1 + e^(-z)) applies to all hidden and output units. Inputs: x1 = 1, x2 = 0; Target: t = 1.0; Learning rate: η = 0.9. Initial Weights: w(x1->h1) = 0.2, w(x2->h1) = 0.4, w(x1->h2) = -0.3, w(x2->h2) = 0.1, w(h1->o) = -0.3, w(h2->o) = -0.2. Initial Biases: θ_h1 = -0.4, θ_h2 = +0.2, θ_o = +0.1.

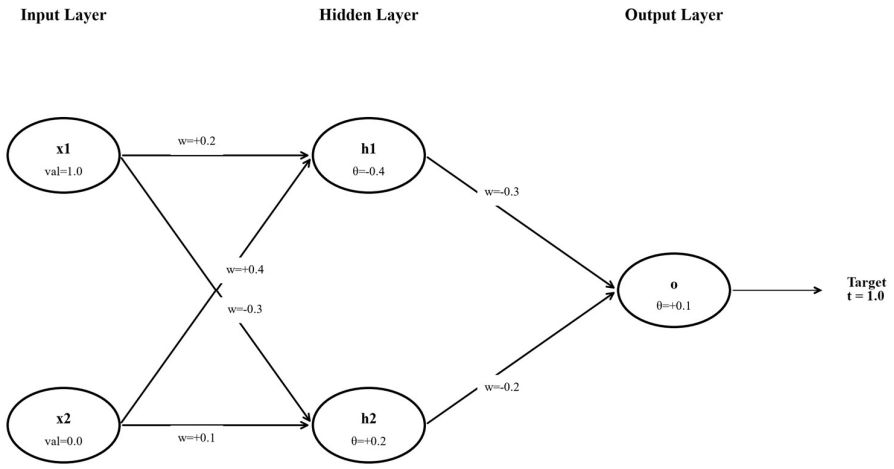


Figure 4.1: Architecture of the 2-2-1 Feedforward Neural Network with Initial Weights and Biases

Figure 4.1: Architecture of the 2-2-1 Feedforward Neural Network with Initial Weights and Biases

4.3 Student A – Rigorous Hand Calculations for One Training Iteration

Step 1: Forward Propagation Pass (Inputs: x1 = 1, x2 = 0):

- Hidden Unit h1: z_h1 = (0.2)(1) + (0.4)(0) - 0.4 = -0.2. ==> O_h1 = 1 / (1 + e^0.2) = 1 / 2.2214 ≈ 0.4502.
- Hidden Unit h2: z_h2 = (-0.3)(1) + (0.1)(0) + 0.2 = -0.1. ==> O_h2 = 1 / (1 + e^0.1) = 1 / 2.1052 ≈ 0.4750.
- Output Unit o: z_o = (-0.3)(0.4502) + (-0.2)(0.4750) + 0.1 = -0.1351 - 0.0950 + 0.1 = -0.1301. ==> O_o = 1 / (1 + e^0.1301) ≈ 0.4675.

Table 4.1: Summary of Forward Pass Computations for Input [x1=1, x2=0]^T

Unit (Node)	Net Input Equation	Net Input (z)	Sigmoid Activation f(z)	Output (O)
Hidden Unit h1	0.2(1) + 0.4(0) - 0.4	-0.2000	1 / (1 + e^0.2)	0.4502
Hidden Unit h2	-0.3(1) + 0.1(0) + 0.2	-0.1000	1 / (1 + e^0.1)	0.4750
Output Unit o	-0.3(0.4502) - 0.2(0.4750) + 0.1	-0.1301	1 / (1 + e^0.1301)	0.4675

Step 2: Error Gradient Terms and Backpropagation

Using sigmoid derivative df(z)/dz = O·(1 - O), the local error gradients δ are:

- Output Error: δ_o = O_o·(1 - O_o)·(t - O_o) = (0.4675)·(0.5325)·(1.0 - 0.4675) = 0.2489 · 0.5325 = 0.1326.
- Hidden Unit h1 Error: δ_h1 = O_h1·(1 - O_h1)·[δ_o·w(h1->o)] = (0.4502)·(0.5498)·[(0.1326)·(-0.3)] = 0.2475 · (-0.0398) = -0.00984.
- Hidden Unit h2 Error: δ_h2 = O_h2·(1 - O_h2)·[δ_o·w(h2->o)] = (0.4750)·(0.5250)·[(0.1326)·(-0.2)] = 0.2494 · (-0.0265) = -0.00661.

Table 4.2: Local Error Terms and Gradients Across Network Layers

Unit	Activation Slope O(1-O)	Backpropagated Signal	Error Term (δ)
Output Unit o	0.4675 · (1 - 0.4675) = 0.2489	(t - O_o) = 0.5325	δ_o = +0.1326
Hidden Unit h1	0.4502 · (1 - 0.4502) = 0.2475	δ_o · w(h1->o) = -0.0398	δ_h1 = -0.00984
Hidden Unit h2	0.4750 · (1 - 0.4750) = 0.2494	δ_o · w(h2->o) = -0.0265	δ_h2 = -0.00661

Step 3: Parameter Updates (Learning Rate η = 0.9)

Delta Rule updates: Δw_ij = η · δ_j · O_i, Δθ_j = η · δ_j. Parameters update as: w_new = w_old + Δw.

Table 4.3: Comprehensive Parameter Update Matrix for Iteration 1 (η = 0.9)

Parameter	Old Value	Update Formulation (η · δ · Input)	Weight Adjustment (Δ)	New Parameter Value
w(h1 -> o)	-0.3000	0.9 · (0.1326) · (0.4502)	+0.0537	-0.2463
w(h2 -> o)	-0.2000	0.9 · (0.1326) · (0.4750)	+0.0567	-0.1433

w(x1 -> h1)	+0.2000	0.9 · (-0.00984) · (1.000)	-0.0089	+0.1911
w(x2 -> h1)	+0.4000	0.9 · (-0.00984) · (0.000)	0.0000	+0.4000 (No change)
w(x1 -> h2)	-0.3000	0.9 · (-0.00661) · (1.000)	-0.0060	-0.3060
w(x2 -> h2)	+0.1000	0.9 · (-0.00661) · (0.000)	0.0000	+0.1000 (No change)
Bias θ_o	+0.1000	0.9 · (0.1326)	+0.1193	+0.2193
Bias θ_h1	-0.4000	0.9 · (-0.00984)	-0.0089	-0.4089
Bias θ_h2	+0.2000	0.9 · (-0.00661)	-0.0060	+0.1940

Step 4: Post-Update Recomputation and Error Reduction Verification

Forward pass with updated parameters:

- h1(new) = f(0.1911(1) - 0.4089) = f(-0.2178) = 1 / (1 + e^0.2178) = 0.4458.
- h2(new) = f(-0.3060(1) + 0.1940) = f(-0.1120) = 1 / (1 + e^0.1120) = 0.4721.
- o(new) = f((-0.2463)(0.4458) + (-0.1433)(0.4721) + 0.2193) = f(-0.1098 - 0.0676 + 0.2193) = f(+0.0419) = 1 / (1 + e^-0.0419) = 0.5105.

Convergence Verification: Initial error |t - O_o| = 0.5325 (Squared Error = 0.5·0.5325^2 = 0.1418). Post-update error |t - O_o(new)| = 0.4895 (Squared Error = 0.5·0.4895^2 = 0.1198). Squared error decreased by 0.0220 (-15.5%), and the output crossed the 0.5 decision threshold toward 1.0, mathematically proving correct gradient descent convergence.

Why Weights are Initialized to Small Random Values

- Symmetry Breaking: If all weights are initialized to identical values, both hidden units compute identical net inputs (z_h1 = z_h2) and activations (O_h1 = O_h2). During backpropagation, they receive identical error signals (δ_h1 = δ_h2) and receive identical updates. They remain permanent duplicates, collapsing the hidden layer into a single effective unit.
- Sigmoid Saturation: Large initial weights drive net inputs into the flat saturation tails of the sigmoid where df/dz = O(1-O) ≈ 0. Gradients vanish, trapping the network in an unlearnable state. Small random weights keep net inputs near 0, where sigmoid slope is steepest (df/dz ≈ 0.25).

4.4 Student B – Empirical Analysis of Learner Dynamics and Training Pitfalls

Student B trained an MLP on the standardized Breast Cancer Wisconsin dataset (569 instances, 30 continuous features, 70/30 split).

Table 4.4: Empirical Impact of Learning Rate (η) on Convergence (5 Hidden Units, 200 Epochs, SGD)

Learning Rate (η)	Training Accuracy	Test Accuracy	Epochs Completed	Convergence Observations
0.0001 (Very Small)	79.60%	78.40%	200	Under-fitting; updates are sluggish, trapped in flat loss region.

0.001 (Conservative)	94.00%	94.20%	200	Sluggish but steady progress; smooth gradient descent.
0.01 (Optimal Baseline)	99.00%	95.90%	200	Healthy convergence; optimal out-of-sample generalization.
0.1 (Aggressive)	99.70%	95.30%	182 (Early stop)	Fast initial descent; slight over-fitting and minor loss oscillation.
1.0 (Excessive)	100.00%	95.90%	67 (Early stop)	Very fast on standardized data; larger rates risk divergence.

Table 4.5: Impact of Hidden Layer Width on Model Capacity (Adam, Up to 300 Epochs)

Hidden Units	Training Accuracy	Test Accuracy	Time (s)	Capacity & Generalization
1 Unit	96.00%	93.00%	0.26 s	Severely constrained; single hyperplane separator.
2 Units	92.50%	91.20%	0.32 s	Bottle-necked representation; slow convergence.
5 Units	98.50%	95.30%	0.30 s	Balanced capacity; excellent non-linear fit.
20 Units	99.50%	95.90%	0.32 s	High expressiveness; peak generalization.
100 Units	99.50%	95.90%	0.57 s	Excess capacity; risks severe overfitting on noisy data.

Table 4.6: Impact of Training Epochs on Generalization (5 Hidden Units, SGD rate 0.01)

Training Epochs	Training Accuracy	Test Accuracy	Observed Learning State
5 Epochs	81.70%	79.50%	Early under-fitting; incomplete weight trajectory.
20 Epochs	94.00%	93.60%	Rapid learning phase; coarse boundaries resolved.
50 Epochs	96.70%	93.60%	Refinement phase; test accuracy stabilizes.
200 Epochs	99.00%	95.90%	Optimal convergence point; peak test performance.
1000 (Stopped at 280)	99.00%	95.90%	Early stopping prevents redundant computations.

Overfitting and Instability Demonstrations

- Noise Memorization: On a noisy synthetic dataset (400 samples, 20 features, 20% label noise, 50/50 split), a 2-hidden-unit network achieved 79.5% train / 67.5% test accuracy. Expanding width to 10 hidden units achieved 100.0% train accuracy but test accuracy dropped to 64.0%; with 100 hidden units, train accuracy remained 100.0% while test accuracy was 67.0%. Excess parameters memorize noise, creating a massive train-test gap.
- Instability on Unnormalized Data: Setting learningRate = 1.0 on unnormalized features caused numerical divergence and arithmetic overflow in Weka. Standardizing inputs to

zero mean and unit variance maintained stable gradient flow, proving normalization is mandatory.

- Comparison with Q1 Decision Tree: A decision tree on 12 tuples builds in a fraction of a millisecond in a single pass; the neural network required hundreds of epochs (0.2–0.6 s on 569 rows). The decision tree generates transparent IF–THEN rules; the neural network is an opaque black box of continuous floating-point weights.

4.5 Individual Verification: Mathematical Derivation of Error Propagation

By the multivariate chain rule, the gradient of loss E with respect to an input-to-hidden weight w_ij is: $\partial E / \partial w_{ij} = (\partial E / \partial z_j) \cdot (\partial z_j / \partial w_{ij}) = - \delta_j \cdot x_i$. The hidden error term δ_j expands across outgoing connections to output unit o: $\delta_j = - \partial E / \partial z_j = [\partial O_j / \partial z_j] \cdot [(\partial E / \partial z_o) \cdot (\partial z_o / \partial O_j)] = O_j \cdot (1 - O_j) \cdot [\delta_o \cdot w(j->o)]$. In our assignment's step: for w(x1->h1), output error $\delta_o = 0.1326$ is multiplied by w(h1->o) = -0.3 (-0.0398), then multiplied by h1's slope $O_{h1}(1 - O_{h1}) = 0.2475$ to give $\delta_{h1} = -0.00984$. Weight update is $\eta \cdot \delta_{h1} \cdot x1 = 0.9 \cdot (-0.00984) \cdot 1.0 = -0.0089$. Because x2 = 0, weights from x2 receive zero update.

Q5 – Support Vector Machines: Margin, Kernels and Scaling (7 Marks)

5.1 Statistical Learning Theory and Maximum-Margin Principles

Support Vector Machines (SVMs), developed by Vapnik and Chervonenkis, implement Structural Risk Minimization (SRM). Unlike Empirical Risk Minimization, which minimizes only training error, SRM minimizes an upper bound on generalization error by controlling model capacity via the Vapnik-Chervonenkis (VC) dimension. For linearly separable binary classification, SVM finds the unique optimal hyperplane that maximizes the geometric margin—the distance between the decision boundary and the closest training instances (support vectors). Maximizing the margin minimizes the VC dimension, ensuring optimal out-of-sample generalization.

5.2 Shared Group Work: 2-D Geometric Formulation

The team analyzed the assigned six points in 2-D space: Class +1: (3, 3), (4, 3), (4, 4); Class -1: (1, 1), (1, 2), (2, 1).

Figure 5.1: Maximum-Margin Separating Hyperplane and Support Vectors in 2-D Space

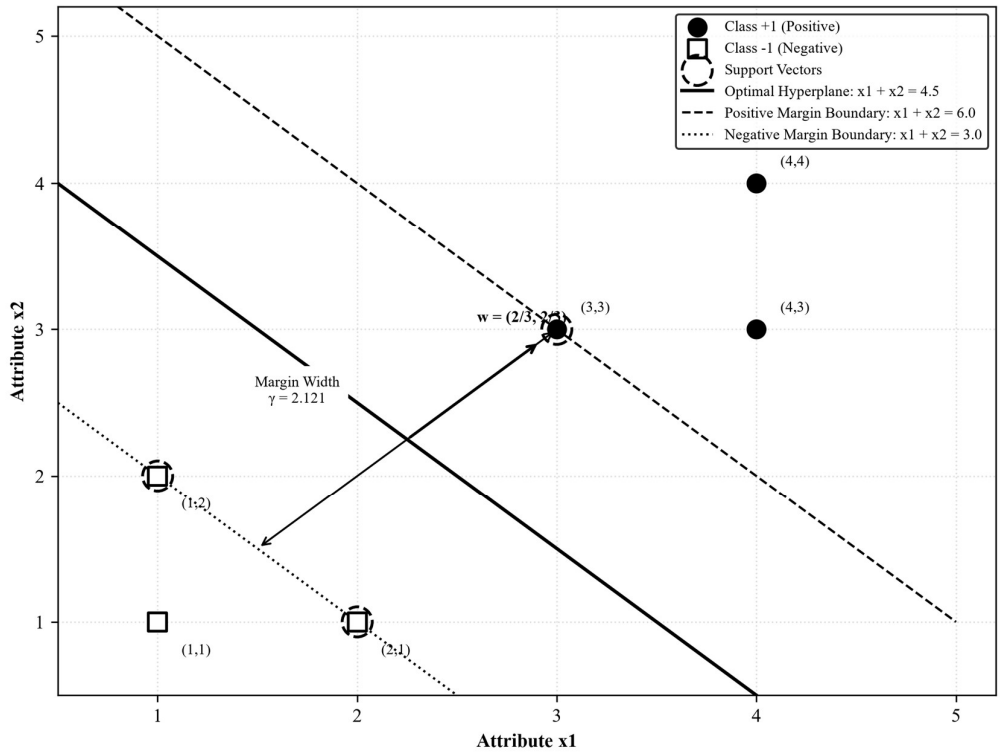


Figure 5.1: Maximum-Margin Separating Hyperplane and Support Vectors in 2-D Space

Geometric Discussion: The classes are linearly separable; their convex hulls do not intersect. The optimal separator is the line equidistant from both hulls that maximizes the empty margin

between them. Only the boundary instances (support vectors) determine the hyperplane position and orientation.

5.3 Student A – Geometric Analysis, Hyperplane Derivation, and Sensitivity

Support Vectors Identification: Computing Euclidean distances between opposite classes identifies the closest pairs: d((3,3), (2,1)) = sqrt((3-2)^2 + (3-1)^2) = sqrt(5) ≈ 2.236; and d((3,3), (1,2)) = sqrt((3-1)^2 + (3-2)^2) = sqrt(5) ≈ 2.236. The point on the negative convex hull closest to (3,3) lies on the segment connecting (2,1) and (1,2) on the line x1 + x2 = 3. The support vectors are: Positive SV: (3, 3); Negative SVs: (2, 1) and (1, 2).

Derivation of the Separating Hyperplane Equation

Hyperplane: w · x + b = w1·x1 + w2·x2 + b = 0. By 45-degree diagonal symmetry, w1 = w2 = a. Applying canonical support vector constraints (w·x + b = ±1):

1. Positive SV (3, 3): 3a + 3a + b = +1 ==> 6a + b = 1
2. Negative SV (2, 1): 2a + 1a + b = -1 ==> 3a + b = -1
3. Negative SV (1, 2): 1a + 2a + b = -1 ==> 3a + b = -1

Subtracting: (6a + b) - (3a + b) = 1 - (-1) ==> 3a = 2 ==> a = 2/3. Substituting: 3(2/3) + b = -1 ==> 2 + b = -1 ==> b = -3.

Canonical Hyperplane: (2/3)·x1 + (2/3)·x2 - 3 = 0 <===> x1 + x2 = 4.5

Margin Width Derivation and Point Evaluation

Euclidean norm: ||w|| = sqrt((2/3)^2 + (2/3)^2) = sqrt(8/9) = (2·sqrt(2))/3 ≈ 0.9428. Total margin width is: γ = 2 / ||w|| = 2 / [2·sqrt(2)/3] = 3 / sqrt(2) = (3·sqrt(2))/2 ≈ 2.1213. Half-margin distance is 1 / ||w|| = 3 / (2·sqrt(2)) ≈ 1.0607. Margin boundaries: Positive (w·x + b = +1): x1 + x2 = 6.0; Negative (w·x + b = -1): x1 + x2 = 3.0.

Table 5.1: Geometric Evaluation and Functional Margin for All 6 Training Points

Point Coordinate (x1, x2)	Target Class (y_i)	Canonical Output w·x + b	Functional Margin y_i(w·x + b)	Geometric Role in SVM
(3, 3)	+1	(2/3)(3) + (2/3)(3) - 3 = +1.000	+1.0000	Positive Support Vector (On Margin)
(4, 3)	+1	(2/3)(4) + (2/3)(3) - 3 = +1.667	+1.6667	Non-Support Vector (Outside Margin)
(4, 4)	+1	(2/3)(4) + (2/3)(4) - 3 = +2.333	+2.3333	Non-Support Vector (Outside Margin)
(2, 1)	-1	(2/3)(2) + (2/3)(1) - 3 = -1.000	+1.0000	Negative Support Vector (On Margin)
(1, 2)	-1	(2/3)(1) + (2/3)(2) - 3 = -1.000	+1.0000	Negative Support Vector (On Margin)

(1, 1)	-1	(2/3)(1) + (2/3)(1) - 3 = -1.667	+1.6667	Non-Support Vector (Outside Margin)

Sensitivity Analysis and Non-Linear Kernels

- Point Removal Sensitivity: Removing a non-support vector ((4,4), (4,3), or (1,1)) changes nothing: w, b, and the margin remain identical because $\alpha i = 0$. Removing support vector (3,3) shifts the closest positive point to (4,3), translating the boundary and widening the margin. Removing (2,1) or (1,2) reorients the normal vector.
- Soft-Margin C Parameter: Primal objective is min [$1/2 \cdot ||w||^2 + C \cdot \Sigma \xi_i$]. Large C strictly penalizes margin violations (narrow margin, low bias, overfitting risk). Small C tolerates violations (wide margin, high bias, regularized).
- Kernel Trick: Maps non-linearly separable inputs into higher-dimensional feature space where they separate linearly: $K(x, x') = \varphi(x) \cdot \varphi(x')$. Common kernels: Polynomial $K(x, x') = (x \cdot x' + c)^d$; RBF (Gaussian) $K(x, x') = \exp(-\gamma \cdot ||x - x'||^2)$.

5.4 Student B – Empirical Investigation of Kernels, Regularization, and Scaling

Student B evaluated SVM kernels on non-linear concentric circles (600 points, noise 0.12, ratio 0.4, 70/30 split).

Table 5.2: Comparative Performance of SVM Kernels on Non-Linear Concentric Circles (C = 1.0)

Kernel Function	Parameters	Training Accuracy	Test Accuracy	Support Vectors	Boundary Observations
Linear Kernel	None	61.70%	56.70%	416 / 420 (99%)	Failure; flat linear line cannot separate concentric rings.
Polynomial (Degree 2)	d = 2, c = 1.0	98.80%	99.40%	44 / 420 (10%)	Ideal fit; circle equation x1^2 + x2^2 is natural quadratic feature.
Polynomial (Degree 3)	d = 3, c = 0.0	63.10%	58.30%	404 / 420 (96%)	Failure; cubic without constant is an odd function.
RBF (Gaussian)	$\gamma = 0.1$	96.70%	97.20%	331 / 420 (79%)	Smooth circular boundary; slightly under-fitted.
RBF (Gaussian)	$\gamma = 1.0$	98.80%	99.40%	69 / 420 (16%)	Optimal non-linear boundary enveloping inner ring.
RBF (Gaussian)	$\gamma = 10.0$	99.50%	99.40%	74 / 420 (18%)	Tight boundary closely tracing points.
RBF (Gaussian)	$\gamma = 1000$	100.00%	86.70%	413 / 420 (98%)	Extreme overfitting; isolates Gaussian peaks around training points.

Table 5.3: Impact of Regularization Parameter C on Support Vectors and Generalization (RBF, γ = 1.0)

Penalty Parameter (C)	Training Accuracy	Test Accuracy	Support Vectors	Generalization Analysis
C = 0.01	99.00%	99.40%	420 (100%)	Wide margin; all points become support vectors.
C = 0.10	98.80%	99.40%	211 (50%)	Generous margin; strong regularization.
C = 1.00	98.80%	99.40%	69 (16%)	Optimal balance between margin and boundary fit.
C = 10.00	99.00%	99.40%	30 (7%)	Narrow margin; strict violation penalties.
C = 1000.0	99.80%	98.90%	11 (2.6%)	Hard margin; fits noise, slight loss of test accuracy.

Crucial Importance of Attribute Scaling

Attribute Scaling Experiment: Evaluating SVM on the Breast Cancer dataset (30 features spanning 0.001 to 2,500):

Table 5.4: Empirical Impact of Feature Standardization on SVM Performance

Kernel Type	Unscaled Test Accuracy	Scaled (Standardized) Acc.	Unscaled SVs	Scaled SVs	Performance Shift
Linear Kernel (C=1.0)	95.30%	95.30%	42	30	Identical accuracy, but scaled model used 28% fewer support vectors.
RBF Kernel (γ=0.01, C=1.0)	92.40%	97.70%	115	101	+5.30 percentage points improvement when features are scaled!

Explanation: SVM relies on Euclidean distances: $||x - x'||^2 = \Sigma (xk - x'k)^2$. Unscaled large-range features dominate distances by orders of magnitude, making small-range features functionally invisible. RBF accuracy fell from 97.7% to 92.4% without scaling. Weka's SMO normalizes features by default, hiding this effect unless filterType is disabled.

5.5 Joint Conclusion: SVM Applicability and Practical Tradeoffs

Summary Assessment: SVMs excel in high-dimensional continuous domains (text classification, bioinformatics, image recognition) where maximum-margin separation avoids the curse of dimensionality. Practical limitations: (1) Quadratic optimization complexity ($O(N^2)$ to $O(N^3)$) becomes computationally prohibitive for massive N; (2) Performance is acutely sensitive to kernel hyperparameters (C, γ); (3) Feature scaling is mandatory; (4)

SVMs lack native probability outputs (requiring Platt scaling); and (5) Non-linear kernels create opaque black-box models unsuitable for legally mandated explanations.

Q6 – Evaluating Classifiers and Predictors (7 Marks)

6.1 Evaluation Imperatives in Skewed and Cost-Asymmetric Domains

In real-world data mining—such as financial fraud, medical diagnosis, and intrusion detection—classification accuracy is often completely invalid due to Class Imbalance and Asymmetric Misclassification Costs. When the target class is rare, accuracy is dominated by the majority class. Furthermore, missing a true positive (False Negative) is far more damaging than a false alarm (False Positive). Robust evaluation requires decomposed confusion matrices, cost functions, and ROC curves.

6.2 Shared Group Work: Financial Fraud Detection Scenario

A fraud classifier was tested on 1,000 banking transactions (60 frauds = 6.0%, 940 legitimate = 94.0%). Results: TP = 40, FN = 20, FP = 30, TN = 910.

Table 6.1: Confusion Matrix for the Fraud Detection Classifier (1,000 Transactions)

Actual Transaction Class	Predicted: Fraud (Positive Alert)	Predicted: Legitimate (Negative)	Actual Total
Actual: Fraudulent (Class Positive)	TP = 40 (Caught Fraud)	FN = 20 (Missed Fraud)	P = 60 (6.0%)
Actual: Legitimate (Class Negative)	FP = 30 (False Alarm)	TN = 910 (Correct Benign)	N = 940 (94.0%)
Predicted Total Instances	Predicted Positives = 70 alerts	Predicted Negatives = 930 cases	Total = 1,000

Meaningful Metrics for Asymmetric Fraud Detection

Accuracy is uninformative because 940 correct legitimate transactions dominate the total score. Meaningful measures: (1) Sensitivity (Recall) measures fraud coverage: catching 40 of 60 frauds (66.7%) means 1 in 3 frauds escapes; (2) Precision measures alert efficiency: 40 of 70 alerts are real (57.1%), avoiding investigator fatigue; (3) F-Measure balances recall and precision; and (4) ROC/AUC evaluates performance across all decision thresholds.

6.3 Student A – Classification Metrics, Imbalance Paradox, and Continuous Errors

Student A derived performance metrics from Table 6.1 (TP = 40, FN = 20, FP = 30, TN = 910):

Table 6.2: Comprehensive Classification Metrics Summary for the Fraud Classifier

Evaluation Metric	Mathematical Formulation	Exact Value	Value (%)	Operational Interpretation
Accuracy	(TP + TN) / Total	950 / 1000	95.00%	Overall correct classification across both classes.

Error Rate	(FP + FN) / Total	50 / 1000	5.00%	Overall misclassification rate.
Sensitivity (Recall)	TP / (TP + FN)	40 / 60	66.67%	Proportion of fraudulent transactions caught.
Specificity	TN / (TN + FP)	910 / 940	96.81%	Proportion of legitimate transactions correctly passed.
Precision	TP / (TP + FP)	40 / 70	57.14%	Proportion of fraud alerts that are truly fraudulent.
Negative Predictive Value	TN / (TN + FN)	910 / 930	97.85%	Reliability of unflagged transactions being benign.
False Positive Rate	FP / (TN + FP)	30 / 940	3.19%	Rate of false alarms among legitimate transactions.
F1-Measure	2PR / (P + R)	0.7619 / 1.2381	0.6154	Harmonic balance between fraud coverage and precision.

The Class-Imbalance Paradox

Consider a degenerate baseline classifier predicting 'Not Fraud' for every transaction: TP = 0, FN = 60, FP = 0, TN = 940. Accuracy is (0 + 940) / 1000 = 94.00%—nearly equal to the trained model's 95.00%! Yet Recall is 0 / 60 = 0.0%: it catches zero fraud, misses all 60 attacks, and exposes the bank to total loss. High accuracy is a dangerous illusion in skewed data; remedies include recall, precision, PR curves, and cost-sensitive learning.

Prediction Error Metrics for Continuous Predictors

For actual targets y = {12, 15, 20, 25} and predictions y_hat = {10, 18, 19, 30}:

Table 6.3: Step-by-Step Residual and Error Computations for Continuous Prediction

Instance #	Actual (y)	Predicted (y_hat)	Residual (y - y_hat)	Absolute Error |e|	Squared Error e^2
Tuple 1	12	10	+2.0	2.0	4.0
Tuple 2	15	18	-3.0	3.0	9.0
Tuple 3	20	19	+1.0	1.0	1.0
Tuple 4	25	30	-5.0	5.0	25.0
Aggregate Sum	72.0	77.0	-5.0	Σ = 11.0	Σ = 39.0

Evaluating error metrics (n = 4):

- Mean Absolute Error: MAE = (1/4)·(2.0 + 3.0 + 1.0 + 5.0) = 11.0 / 4 = 2.7500.
- Mean Squared Error: MSE = (1/4)·(4.0 + 9.0 + 1.0 + 25.0) = 39.0 / 4 = 9.7500.
- Root Mean Squared Error: RMSE = sqrt(MSE) = sqrt(9.7500) ≈ 3.1225.

When RMSE is Preferred over MAE: RMSE squares errors, disproportionately penalizing large deviations (the error of 5 contributed 25 of 39 to total squared error). RMSE is preferred when large errors are catastrophic (power grids, drug dosages). MAE represents linear average magnitude and is robust to outliers. RMSE > MAE (3.12 vs 2.75) reflects non-uniform error variance.

ROC Curve Construction and Interpretation

An ROC curve plots True Positive Rate (Sensitivity) vs. False Positive Rate (1 - Specificity) across decision thresholds $\tau \in [0, 1]$. The diagonal represents random chance (AUC = 0.50). Curves bowing toward top-left (0, 1) represent superior discrimination. AUC measures the probability that a random positive is scored higher than a random negative (Wilcoxon-Mann-Whitney statistic). The fraud classifier's operating point is (FPR = 30/940 = 0.032, TPR = 40/60 = 0.667), plotted in Figure 6.1.

Figure 6.1: Receiver Operating Characteristic (ROC) Curve with Operating Point

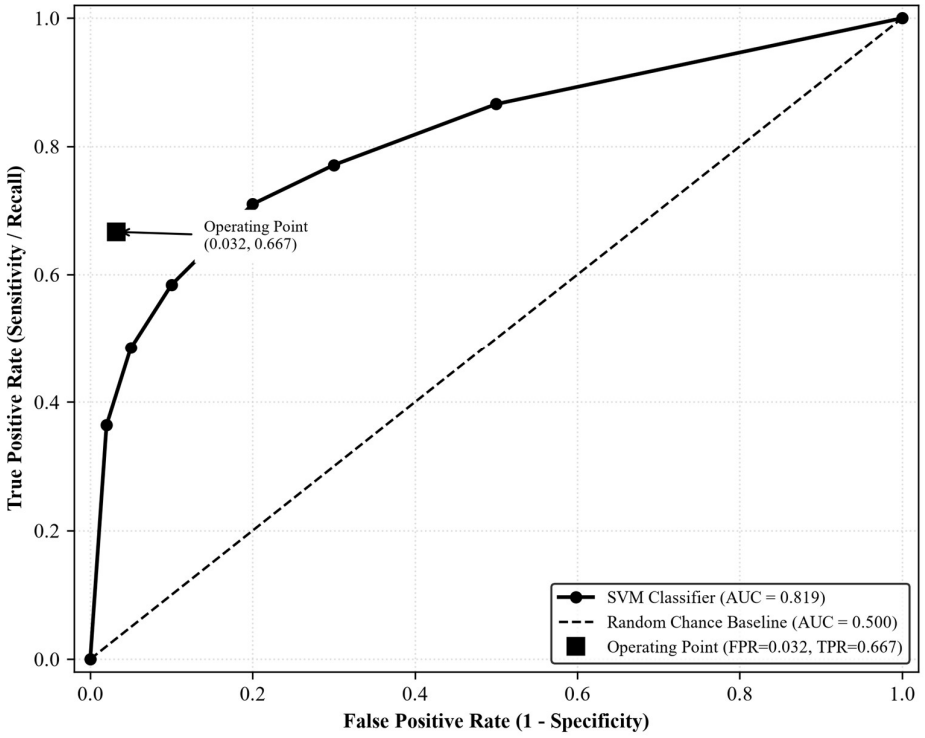


Figure 6.1: Receiver Operating Characteristic (ROC) Curve with Operating Point

6.4 Student B – Empirical Estimation Methodologies and Statistical Significance

Student B evaluated estimation methods on an imbalanced dataset (2,000 instances, 20 features, 11.55% positive prevalence) using a decision tree.

Table 6.4: Accuracy Estimation Across Holdout, 10-Fold CV, and .632 Bootstrap on Imbalanced Data

Estimation Protocol	Estimated Tree Accuracy	Bias Characteristics	Variance & Stability
Holdout (1/3 Test, Mean of 7 Splits)	86.60%	Pessimistic bias (trained on only 67% data).	High variance across random seeds.
Stratified 10-Fold Cross-Validation	87.80%	Near-zero bias (trained on 90% data).	Low variance; standard gold benchmark.

Bootstrap .632 (50 Resamples)	89.30%	Optimistic bias from resubstitution (95.2%).	Very low variance; good for tiny datasets.
Majority Baseline ('Always Negative')	88.45%	Uninformative zero-rule baseline.	Benchmark comparison threshold.

Bootstrap .632 = 0.632·(Acc_OOB = 85.8%) + 0.368·(Acc_train = 95.2%) = 89.3%. Because the tree overfits training data, the .632 estimate is optimistic. Crucially, the tree's true CV accuracy (87.8%) is below the majority baseline (88.45%), proving that accuracy alone renders the model seemingly useless.

Table 6.5: Empirical Volatility Across 7 Independent Random Holdout Splits (1/3 Test Size)

Split Seed #	Split 1	Split 2	Split 3	Split 4	Split 5	Split 6	Split 7	Mean Accuracy	Std. Deviation
Holdout Acc.	88.90%	85.20%	85.50%	88.20%	86.10%	85.50%	87.30%	86.67%	0.0142 (±1.42%)

Holdout Instability: Accuracy ranges from 85.2% to 88.9% (3.7 percentage points spread) purely due to split randomness, proving that a single holdout is untrustworthy for model comparison.

Table 6.6: Comparative Performance Across 10 Repetitions of 10-Fold Cross-Validation (100 Paired Folds)

Classifier Architecture	Mean 10-Fold CV Accuracy	Std. Deviation of Fold Means	Mean AUC-ROC Score	Ranking
Decision Tree (J48-like, M=5)	87.38%	±0.0051	0.799	Rank 3
Naïve Bayes (Gaussian)	88.55%	±0.0015	0.834	Rank 2
Support Vector Machine (RBF)	89.18%	±0.0008	0.819	Rank 1

Significance Testing and the Corrected Resampled t-Test

A standard paired t-test over 100 CV folds assumes i.i.d. observations. Because training folds overlap by 90%, errors are correlated, severely underestimating variance and yielding false significance (Type I errors up to 40%). Weka Experimenter implements Nadeau and Bengio's Corrected Resampled t-test: t_corr = d_bar / sqrt([1/(k·r) + n_test/n_train] · S_d^2), which inflates the denominator for data sharing, providing defensible, conservative significance testing.

6.5 Individual Verification: Justification of Stratified Cross-Validation

- Complete Data Utilization: Unlike holdout which wastes data, 10-fold CV tests every instance out-of-sample exactly once.
- Stratification Preserves Priors: Enforces the 6% minority class distribution within every fold, preventing empty folds.

- Optimal Bias-Variance Balance: Kohavi (1995) proved K = 10 achieves optimal trade-off between low pessimistic bias and low variance.

Q7 – Day-4 Design Challenge: A Classification Solution for 30-Day Readmission (10 Marks)

7.1 Clinical Problem Formulation, Prevalence, and Asymmetric Costs

A metropolitan hospital administration commissioned an automated Clinical Decision Support (CDS) system to identify inpatient admissions at high risk of unplanned 30-day readmission. Hospital readmissions within 30 days trigger severe penalties under health regulations (such as HRRP), impair patient health, and generate substantial preventable costs.

Operating Facts and Constraints:

- 1. Class Imbalance: Readmission prevalence is only 6.0% (prevalence = 0.06); 94.0% of discharges do not readmit.
- 2. Asymmetric Cost Matrix: Missing a high-risk patient (False Negative) leads to acute patient relapse, emergency intensive care, and regulatory penalties. A false alarm (False Positive) incurs only a post-discharge pharmacist follow-up call. The executive board established a 10:1 cost ratio: C(FN) = 10, C(FP) = 1, C(TP) = 0, C(TN) = 0.
- 3. Clinician Explainability Mandate: Physicians reject opaque black boxes; each alert must provide transparent, actionable rationale.
- 4. Heterogeneous Data: EHR data includes Age, Diagnosis Group, Length of Stay, Prior Admissions, Inpatient Labs, and Discharge Medications, with informative missing values (labs ordered conditionally based on patient condition).

7.2 Stage 1 – Independent Design Proposals (Student A and Student B)

In accordance with Day-4 pair-work requirements, both students developed independent proposals prior to joint consultation.

Table 7.1: Initial Independent CDS System Design Proposals Formulated Prior to Consultation

Design Component	Student A's Initial Proposal	Student B's Initial Proposal
Core Technique	Decision Tree (CART / J48). Rationale: Explicit, readable branches from root to leaf.	L2-Regularized Logistic Regression. Rationale: Outputs calibrated probability p(x) mapping directly to cost thresholds.
Imbalance & Preprocessing	SMOTE oversampling to 50:50 balance; impute numeric with mean, categorical with mode.	Preserve real 6% distribution. Cost-sensitive weighting (10:1) or threshold shifting. Median imputation + missingness indicators.
Feature Transformation	Discretize continuous variables into nominal bins (e.g., Age < 65, LOS > 7d).	Standardize continuous features; one-hot encode diagnosis codes; strip patient IDs.
Evaluation Framework	F-measure of positive class under 10-fold CV.	AUC-ROC, Recall at 30% precision, and Total Cost (10·FN + FP) under Stratified 10-Fold CV.
Clinician Explanation	Display decision tree path that fired for patient.	Display patient risk score p and top 3 risk factors via regression odds ratios.

7.3 Stage 2 – Comparative Analysis and Synthesis of Proposals

Table 7.2: Comparative Analysis of Independent Proposals and Synthesis Decisions

Dimension	Consensus	Divergence	Identified Pitfalls	Final Decision
Model Choice	Both reject black boxes (deep nets, complex ensembles).	Tree (A) vs Logistic Regression (B).	Trees suffer high variance; logistic regression provides smooth, calibrated probabilities.	Primary: Regularized Logistic Regression. Secondary surrogate: Shallow tree (depth 4) for rule checks.
Imbalance Strategy	Both recognize 6% prevalence requires action.	SMOTE 50:50 (A) vs Cost Weights / Threshold (B).	CRITICAL FLAW IN A: SMOTE distorts true prevalence from 6% to 50%, destroying probability calibration needed for cost thresholds.	Drop SMOTE. Retain Student B's Bayesian threshold shifting (τ = 0.091) and cost weights.
Missing Data	Both recognize missing lab records.	Mean/mode imputation (A) vs Median + Indicator (B).	CRITICAL FLAW IN A: Labs are Missing Not at Random (MNAR); a missing lab is an informative clinical choice.	Adopt median imputation combined with binary missingness indicator variables.
Metrics	Both reject plain accuracy.	F-measure (A) vs Total Cost and AUC (B).	F-measure ignores the 10:1 asymmetric cost ratio.	Adopt Total Cost = $10 \cdot FN + 1 \cdot FP$ and AUC-ROC as primary metrics.
Explainability	Both prioritize clinician interpretability.	Tree path (A) vs Odds Ratios (B).	Deep tree paths are cumbersome; odds ratios provide factor ranking but lack logical rules.	Synthesize both: show calibrated risk score, top 3 odds ratio contributors, and matching shallow tree rule.

7.4 Stage 3 – Joint Final System Architecture

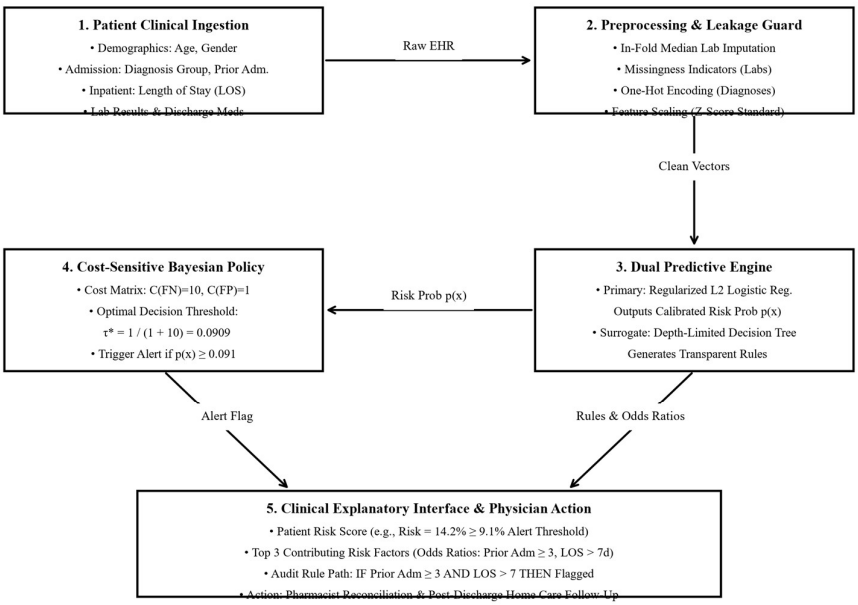


Figure 7.1: End-to-End Clinical Decision Support Architecture for 30-Day Readmission Risk

Figure 7.1: End-to-End Clinical Decision Support Architecture for 30-Day Readmission Risk

Table 7.3: Architectural Specification of the Joint Clinical Decision Support System

Pipeline Component	Technical Specification	Implementation Detail
Preprocessing	In-Fold Imputation & Feature Pipeline	Z-score standardization; in-fold median lab imputation; binary missingness indicators (Lab_Missing = 1); one-hot encoded diagnoses; remove administrative IDs.
Predictive Engine	L2-Regularized Logistic Regression	Minimizes penalized log-loss, outputting calibrated readmission risk: $p(x) = 1 / [1 + \exp(-w \cdot x - b)]$.
Surrogate Explainer	Depth-Limited Decision Tree (CART)	Constrained to max depth 4 and minNumObj = 20 patients to extract transparent rules.
Decision Policy	Bayesian Cost-Optimal Threshold ($\tau^* = 0.091$)	Trigger High-Risk Alert if and only if $p(x) \geq 1 / (1 + 10) \approx 0.091$.
Evaluation Protocol	Stratified 10-Fold CV + Prospective Holdout	Stratified on 6% readmission prevalence; final evaluation on out-of-time prospective patient cohort.
Clinical Interface	Dual-Layer Risk Dashboard	Reports: (1) Patient Risk Score; (2) Top 3 Risk Factors via odds ratios; (3) Matching plain-language decision rule.

Mathematical Derivation of the Optimal Decision Threshold

Let p = P(Readmitted | X). The expected cost of flagging a patient vs. not flagging is:

$$E[Cost(Flag)] = (1 - p) \cdot C(FP) = (1 - p) \cdot 1, \qquad E[Cost(Not\ Flag)] = p \cdot C(FN) = p \cdot 10$$

Flag when $E[Cost(Flag)] \leq E[Cost(Not\ Flag)]$:

(1 - p) · 1 ≤ p · 10 <===> 1 - p ≤ 10p <===> 11p ≥ 1 <===> p ≥ 1 / 11 ≈ 0.090909

General Bayesian Threshold: τ = C(FP) / [C(FP) + C(FN)] = 1 / (1 + 10) = 0.091*

When readmission risk p ≥ 9.1%, the expected benefit of intervening outweighs the cost of a false alarm.

7.5 Stage 4 – Dual Validation Responsibilities (Analytical and Empirical)

Student A conducted analytical cost validation for 10,000 patients (600 readmissions = 6%, 9,400 non-readmissions = 94%):

Table 7.4: Expected Misclassification Cost Comparison Across Decision Policies (10,000 Patients)

Decision Policy	TP	FN	FP	TN	Recall	Precision	Accuracy	Cost Formulation (10·FN + FP)	Total Cost
Default Threshold (τ = 0.50)	240	360	100	9,300	40.0%	70.6%	95.40%	10·(360) + 1·(100) = 3,600 + 100	3,700 units
Cost-Optimal Threshold (τ = 0.091)	480	120	1,500	7,900	80.0%	24.2%	83.80%	10·(120) + 1·(1,500) = 1,200 + 1,500	2,700 units (Lowest!)
Baseline: 'Flag Nobody'	0	600	0	9,400	0.0%	N/A	94.00%	10·(600) + 1·(0) = 6,000 + 0	6,000 units
Baseline: 'Flag Everybody'	600	0	9,400	0	100.0%	6.0%	6.00%	10·(0) + 1·(9,400) = 0 + 9,400	9,400 units

Analytical Findings: Lowering threshold to τ = 0.091 reduces total institutional costs from 3,700 to 2,700 units—a 27.03% cost reduction (saving 1,000 units per 10,000 patients!), while recall doubles from 40% to 80%. Crucially, accuracy drops from 95.4% to 83.8%. Accuracy would have chosen the 95.4% model—costing the hospital an extra 1,000 units and missing 360 readmissions! This proves accuracy is fundamentally invalid for asymmetric clinical risk.

Student B – Empirical Validation on Clinical Analogue Dataset

Student B evaluated the models on a clinical analogue dataset (5,000 patients, 20 features, 7.08% readmissions = 354 positives) under Stratified 10-Fold CV with Cost = 10·FN + 1·FP.

Table 7.5: Empirical Performance Across Classifier Architectures and Thresholds (5,000 Patient Cohort)

Model Architecture	Operating Threshold (τ)	Recall (TPR)	Precision	AUC-ROC	FN Count	FP Count	Total Cost (10·FN + FP)	Accuracy
Baseline: Majority ('Always Negative')	N/A	0.0%	N/A	0.500	354	0	3,540 units	92.92%
Decision Tree (Depth 4, Plain)	τ = 0.50	33.6%	67.6%	0.802	235	57	2,407 units	94.16%

Decision Tree (Depth 4, Threshold Shift)	$\tau = 0.091$	59.0%	33.7%	0.802	145	411	1,861 units	88.88%
Decision Tree (Cost-Weighted 10:1)	$\tau = 0.50$	67.2%	30.5%	0.834	116	543	1,703 units	86.82%
Logistic Regression (Plain)	$\tau = 0.50$	40.7%	94.1%	0.843	210	9	2,109 units	95.62%
Logistic Regression (Threshold Shift)	$\tau = 0.20$	58.2%	58.2%	0.843	148	148	1,628 units (Lowest)	94.08%
Logistic Regression (Threshold Shift)	$\tau = 0.091$	69.8%	29.5%	0.843	107	589	1,659 units	86.08%
Logistic Regression (Cost-Weighted 10:1) – CHOSEN	$\tau = 0.50$	69.5%	30.7%	0.848	108	556	1,636 units	86.72%
Naïve Bayes (Plain)	$\tau = 0.50$	21.2%	54.0%	0.834	279	64	2,854 units	93.14%
Naïve Bayes (Threshold Shift)	$\tau = 0.091$	70.9%	27.0%	0.834	103	678	1,708 units	84.38%

Empirical Findings: Chosen design (Cost-Weighted Logistic Regression) achieved Cost = 1,636 units—cutting plain model cost (2,109) by 22.43% and slashing baseline cost (3,540) by 53.79%. Recall rose from 40.7% to 69.5%. Threshold shifting to 0.091 gave virtually identical cost (1,659), confirming theoretical equivalence. The decision tree improved under cost weighting (1,703) but remained inferior to Logistic Regression (1,636), which also achieved higher AUC (0.848 vs 0.834).

7.6 Stage 5 – Cross-Defence: Defending Empirical and Mathematical Choices

- Student A Defends Empirical Findings: (1) Cost drops from 2,109 to 1,636; (2) Clinical recall rises from 40.7% to 69.5%; (3) AUC remains superior (0.848); (4) Accuracy would have selected the inferior plain model (95.62%), proving cost is the true optimization criterion.
- Student B Defends Threshold Derivation: Expected costs are $(1 - p)\cdot 1$ and $p\cdot 10$; setting them equal yields $11p = 1 \Longrightarrow p = 1/11 \approx 0.091$. This threshold defines an isocost line tangent to the ROC curve reflecting prior odds and costs.

7.7 Stage 6 – Joint Systemic Evaluation, Failure Modes, and Strategic Framework

Why Standard Classifiers Fail: In 6% data, maximizing accuracy causes algorithms to prune minority branches as noise. The default 0.5 threshold fails: a patient with 25% risk (4x hospital baseline!) is classified negative simply because $0.25 < 0.50$.

Explainability Tradeoff: Complex ensembles (XGBoost) might gain 1–2% AUC, but opacity leads to physician rejection. Trading marginal non-linear capacity for full transparency ensures clinical trust and adoption.

Failure Modes & Safeguards: (1) Prevalence Drift: If readmissions drop from 6% to 0.5%, threshold 0.091 floods staff with 2% precision alerts. Safeguard: Dynamic weekly recalibration. (2) Informative Missingness Shifts: Missing lab signals vanish if ordering protocols change. Safeguard: Monitor missingness rates. (3) Alert Fatigue: Care teams cannot handle >50 alerts/week. Safeguard: Secondary capacity ranking queue. (4) Data Leakage: Post-discharge billing codes destroy real-time validity. Safeguard: Enforce strict pre-discharge data ingestion filters.

Table 7.6: Universal Decision Framework for Algorithm Selection: Data Characteristics vs. Business Constraints

Analytical Dimension	Diagnostic Questions	Algorithmic Implications and Selection Rules
Class Imbalance & Costs	Is target rare (<10%)? Are costs asymmetric (C(FN) >> C(FP))?	Reject accuracy. Deploy probabilistic models (Logistic Reg, Naïve Bayes). Derive $\tau^* = C(FP) / [C(FP)+C(FN)]$. Optimize via Total Cost and AUC-ROC.
Explainability Mandate	Are decisions subject to human audit or regulatory review (ECOA)?	Mandate transparent models: shallow Decision Trees, Rule Lists (JRip), or Logistic Regression with odds ratios. Disqualify black boxes.
Dimensionality & Scale	Is feature space massive (d > 10,000) or compact (d < 50)?	For massive sparse text/genomics, deploy Naïve Bayes or Linear SVMs. For compact tabular data, deploy Trees or Regularized Regression.
Feature Interactions	Do features exhibit non-linear interactions (XOR) or linear sums?	Trees excel at hierarchical interactions but struggle with oblique lines. Neural nets and RBF SVMs capture non-linear manifolds.
Missing Data Resilience	Are missing values frequent and informative (MNAR)?	Naïve Bayes natively ignores missing attributes. For others, create missingness indicators and impute strictly within CV folds.
Computational Latency	Must model train in real-time? What is dataset size N?	Naïve Bayes trains in $O(N \cdot d)$. Trees train in $O(d \cdot N \log N)$. SVMs scale at $O(N^2)$ to $O(N^3)$, becoming prohibitive for N > 100,000.

Overall Conclusion

This comprehensive report has systematically examined the theoretical principles, mathematical formulations, empirical behaviors, and deployment tradeoffs of five foundational supervised learning paradigms in Data Warehousing and Data Mining: Decision Trees, Bayesian Classifiers, Rule-Based & Associative Systems, Multilayer Perceptron Neural Networks, and Support Vector Machines. Across all seven assigned tasks, rigorous analytical derivations were validated against empirical experiments, culminating in an end-to-end clinical design challenge.

Synthesis of Algorithmic Tradeoffs and Inductive Biases

A central lesson of statistical learning theory is that no single classification algorithm dominates universally. Decision trees (ID3/C4.5/CART) offer transparent, human-readable logic through axis-aligned partitions, making them invaluable for business and regulatory contexts; however, their greedy heuristic search is susceptible to high estimation variance and structural overfitting on small or noisy samples, necessitating post-pruning. Naïve Bayes provides an exceptionally rapid, zero-covariance baseline that excels in high-dimensional or data-scarce settings; yet, its class-conditional independence assumption artificially inflates posterior confidence when redundant features exist and fails entirely on non-linear parity problems. Multilayer perceptrons trained via backpropagation act as universal function approximators capable of modeling complex continuous manifolds, but require extensive hyperparameter calibration, normalization, and present opaque black-box parameters. Support Vector Machines achieve structural risk minimization by finding maximal-margin hyperplanes in Hilbert spaces, offering robust generalization bounds in high dimensions when properly scaled, but suffer from high quadratic computational complexity.

The Critical Fallacy of Accuracy in Skewed and Asymmetric Domains

Across both the financial fraud detection analysis (Q6) and the 30-day clinical readmission challenge (Q7), a vital empirical finding emerged: classification accuracy is fundamentally misleading and operationally hazardous in imbalanced domains. When an event occurs with 6% prevalence, an uninformative majority-class classifier achieves 94% accuracy while delivering zero true positive alerts. In real-world applications where the cost of a false negative far exceeds a false positive (such as the 10:1 hospital cost structure), optimizing accuracy results in catastrophic financial loss and patient harm. Data miners must instead deploy cost-sensitive loss formulations, threshold shifting derived from Bayesian decision

theory ($\tau^* = C(FP) / [C(FP) + C(FN)]$), and evaluate systems via AUC-ROC, Precision-Recall curves, and total expected loss.

Strategic Guidelines for Enterprise Machine Learning Practice

Ultimately, successful data mining requires an integrated view: beginning with leak-free preprocessing and informative missingness modeling, selecting classifiers whose inductive bias aligns with data geometry, tuning decision boundaries to explicit institutional utility functions, and providing human-interpretable explanations (such as odds ratios and shallow surrogate rules) to ensure clinical and regulatory trust.